

1 3D Tools

TikZ Library `3dtools`

```
\usetikzlibrary{3dtools} % LATEX and plain TEX
\usetikzlibrary[3dtools] % ConTEXt
```

This library provides additional tools to create 3d-like pictures. It is a collection of reasonably working tools, which however is not streamlined, and may be subject to substantial changes if the library ever happens to get further developed or published. This library loads the `3d`, `decorations` and `fp` libraries.

TikZ has the `3d` and `perspective` libraries which deal with the projections of three-dimensional drawings. In addition there exist excellent packages like `tikz-3dplot`. The purpose of this library is to provide some means to manipulate the coordinates. It supports linear combinations of vectors, vector products and scalar products. It also supports “physical” coordinates and means to transform coordinates from one orthonormal basis to another orthonormal basis.

Note: Hopefully this library is only temporary and its contents will be absorbed in slightly extended versions of the `3d` and `calc` libraries.¹ The cleanest way will be to record a screen depth when “saving” a coordinate with TikZ. Some limited support of such a functionality is provided in this library, see section 1.6. However, a full-fledged realization would require changes at the level of `tikz.code.tex` and can only be done consistently if the maintainer(s) of TikZ make some fundamental changes. Even if this happens, dealing with node anchors and so on, which so far are intrinsically two-dimensional (see, however, <https://github.com/ZhiyuanLck/dnnplot> for an impressive counterexample) will be nontrivial because one will have to specify the orientation in 3d as far as supported. Note also that it is quite conceivable that the viewers in the future will be able to achieve 3d ordering, so, in a way, recording the screen depth (see the `screendepth` function below) will become almost mandatory at a given point.

Filing a bug report or placing a feature request. This library is currently hosted under <https://github.com/marmotghost/tikz-3dtools>.

Why is this library not on CTAN? First of all, ideally this library is only temporary. Secondly, it is also under construction. There are many things that users may want to be able to use, such as spherical coordinates, and which are not supported at all. Adding support for such features may require syntax changes, thus making currently working code invalid. If this is the case, it would not be helpful if the library had already been made “official”.

Code examples. This manual contains a number of code examples. They do not consist of complete L^AT_EX documents but only excerpts. Almost all of them require the `3dtools` library but they may require additional libraries that do not get loaded by the `3dtools` library. All libraries that are required to compile a code example are indicated. Each code example can be embedded in a minimal example, which can have the form

<code>\documentclass[tikz]{standalone}</code>	also loads TikZ
<code>\usetikzlibrary{3dtools,...}</code>	libraries as indicated
<code>\begin{document}</code>	
<code><code></code>	code from the example
<code>\end{document}</code>	

It should be also possible to embed the code in generic L^AT_EX documents as long as in the preamble TikZ gets loaded (e.g. with `\usepackage{tikz}`) and the relevant libraries get also loaded. Note that although above ConT_EXt gets mentioned this library has not been tested with ConT_EXt. The only reason why ConT_EXt gets mentioned is that this gets automatically added by `pgfmanual.code.tex`, which gets loaded to prepare this document.

1.1 Coordinate computations

The `3dtools` library has some options and styles for coordinate computations and manipulations.

¹Note that an earlier version of the library had been uploaded to CTAN by the maintainer of TikZ at that time. The maintainer did not notify the author of the library, let alone ask for permission. Since the library was in an even worse shape at that time, it got removed by the author. For further information on this incident please consider contacting CTAN (rather than simply believing in chat messages, which are often not too accurate).

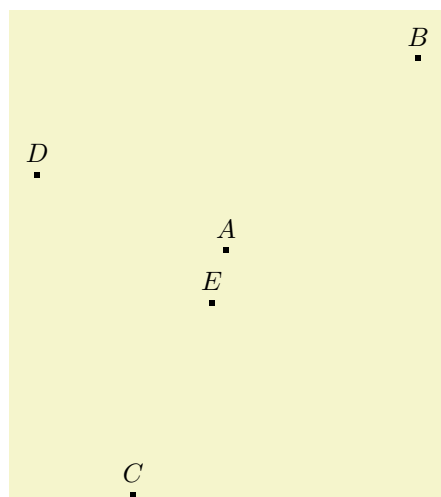
`/tikz/3d parse` (no value)

Parses an expression and inserts the result in form of a coordinate.

`/tikz/3d coordinate` (no value)

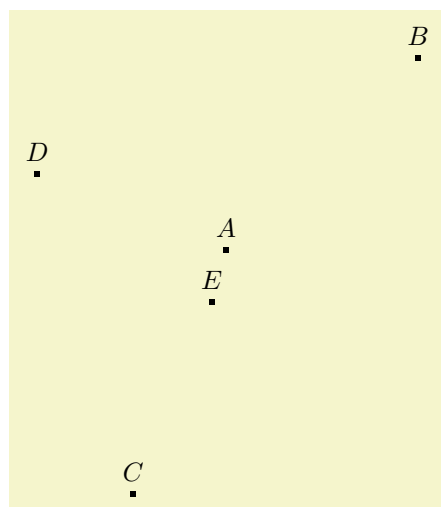
Allow one to define a 3d coordinate from other coordinates.

Both keys support both symbolic and explicit coordinates.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}
\path (1,2,3) coordinate (A)
(2,3,-1) coordinate (B)
(-1,-2,1) coordinate (C)
[3d parse={0.25*(1,2,3)x(B)}]
coordinate(D)
[3d parse={0.25*(C)x(B)}]
coordinate(E);
\path foreach \X in {A,...,E}
{(\X) node[fill,inner sep=1pt,
label=above:$\X$]{};};
\end{tikzpicture}
```

Notice that, as of now, only the syntax `\path (1,2,3) coordinate (A);` works, yet the syntax `\coordinate (A) at (1,2,3);` does *not* work, but leads to error messages.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}
\path (1,2,3) coordinate (A)
(2,3,-1) coordinate (B)
(-1,-2,1) coordinate (C)
[3d coordinate={ (D)=0.25*(1,2,3)x(B) },
3d coordinate={ (E)=0.25*(C)x(B) },
3d coordinate={ (F)=(A)-(B) },];
\path foreach \X in {A,...,E}
{(\X) node[fill,inner sep=1pt,
label=above:$\X$]{};};
\end{tikzpicture}
```

The actual parsing are done by the function `\pgfmathtdparse` that allows one to parse 3d expressions. The supported vector operations are + (addition +), - (subtraction -), * (multiplication of the vector by a scalar), x (vector product $\times$) and o (scalar product). There is limited support for ordinary PGF parsings, which are to be wrapped into `[...]`. Admittedly, this function is currently very poor. Enhancing it will be a top priority if this is ever to become a CTAN library/package.

`\pgfmathtdparse{<x>}`

Parses 3d expressions.

`TDx("vector")`

Yields the x -component of a 3d expression.

`TDy("vector")`

Yields the y -component of a 3d expression.

`TDz("vector")`

Yields the z -component of a 3d expression.

`TDunit("vector")`

Yields the result of the expression normalized to unity. If the length of the expression is too close to zero, a warning is issued and the unnormalized vector gets returned.

`screendepth("vector")`

Yields distance a coordinate is above (positive) or below (negative) the screen. The values are only really meaningful if the user has installed some reasonable view. The larger the screen depth of a point is, the closer is the point to the observer. The function reconstructs the 3-bein² from lengths like `\pgf@xy` and so on, so the function is independent of the tool that is employed to install a view (cf. section 1.2). The screen depth is crucial to decide whether a given object is in front of other objects, or behind.

`tddistance("point 1","point 2")`

Yields the distance between *point 1* and *point 2*. You have to keep the parentheses, e.g. `tddistance("(P)","(Q)")` works if you defined the points (P) and (Q), but not `tddistance("P","Q")`;

`nscreenx`

Yields the x -component of the normal on the screen.

`nscreeny`

Yields the y -component of the normal on the screen.

`nscreenz`

Yields the z -component of the normal on the screen.

`x2d`

Yields the x -component of a symbolic coordinate on the screen.

`y2d`

Yields the y -component of a symbolic coordinate on the screen.

In order to pretty-print the result one may want to use `\pgfmathprintvector`, and use the math function TD for parsing.

`\pgfmathprintvector{\langle x \rangle}`

Pretty-prints vectors.

$$0.2 \vec{A} - 0.3 \vec{B} + 0.6 \vec{C} = (-1, -1.7, 1.5)$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("0.2*(A)
-0.3*(B)+0.6*(C)")}%
$0.2\,\vec A-0.3\,\vec B+0.6\,\vec C
=(\pgfmathprintvector\pgfmathresult)$
```

The alert reader may wonder why this works, i.e. how would TikZ “know” what the coordinates A , B and C are. It works because the coordinates in TikZ are global, so they get remembered from the above example.

Named coordinates. TikZ only remembers where a named node or coordinate is on the canvas. To know the 3d point behind it, this library records, for every named node and coordinate, its canvas position together with its screen depth, i.e. its distance from the screen (see `screendepth` below). For any non-degenerate view, these two determine the point. The depth is tracked through

- (x, y, z) , (x, y) (which TikZ reads as $(x, y, 0)$) and polar coordinates with dimensionless radii;

²A 3-bein or dreibein is a German word that stands for a local frame, its literal translation is something like three-leg. Like the term eigenvector this is a foreign word that, to the best of my knowledge, has no commonly used English counterpart.

- named coordinates and anchors: the anchors of a node have the depth of the node, since nodes are flat on the screen;
- `++(...)`, `+(...)`, `shift={(...)}` and local frames (section 1.3);
- sums, differences, factors and partway modifiers `!\langle t \rangle!` of the `calc` library;
- `coordinate[pos=\langle t \rangle]` on straight `--` segments.

So `\coordinate (P) at (1,2,3);`, `(A) ++(1,0,0) coordinate (P)` and `($A)!0.5!(B)$ coordinate (M)` all give the expected points. A coordinate in canvas units, such as `(1cm,2cm)` or `(30:1cm)`, lies in the plane parallel to the screen through the origin of the coordinate system in force. The depth is *not* tracked for distance and projection modifiers (`!1cm!`, `!(C)!`) and rotations in `calc`, perpendicular coordinates `(A|-B)`, `pos=` on curves, arcs and `-|` or `|-` segments, intersections, and coordinates defined in `canvas is xy plane at z=0` and its relatives or with `reset cm`. For such coordinates TD issues a warning and falls back to the text with which the coordinate was defined, i.e. to the `\tikz@dc1@coord@{coord}` macro of TikZ, as earlier versions of this library did.

Inside the picture in which a coordinate was defined, and under the same view, `TD("(P)")` yields P in the coordinate system in force: in a scope with `shift={ (1,0,0) }` a coordinate defined outside of it appears shifted by $(-1,0,0)$, and inside a local frame TD yields local coordinates. This is what `pics` and path constructions that take named points need. Elsewhere, i.e. in node texts, outside of pictures and under a different view, `TD("(P)")` yields P in the coordinate system in which P was defined. The results are rounded to five decimals and accurate to about 10^{-5} (10^{-4} in scaled pictures); a coordinate that was given as a list of plain numbers is returned exactly when TD is evaluated where it was defined. The `3d coordinate` key stores the computed vector itself, and TD of such a coordinate always returns that vector, which is what one wants for directions. Since the recorded position is used, a coordinate defined with random numbers yields the same value each time.

$$\vec{R} = (1.03, 0.49, 0.4)$$

$$\vec{R} = (1.03, 0.49, 0.4)$$

```
\usetikzlibrary {3dtools}
\begin{tikzpicture}
\path[overlay] (0.1+rnd,0.1+rnd,0.1+rnd)
coordinate (R);
\def\ppv{\pgfmathprintvector\pgfmathresult}
\draw[red] (R) circle[radius=0.1];
\node at (0,1)
{\pgfmathparse{TD("(R)")}%
$\vec R=(\ppv)$};
\draw[blue] (R) circle[radius=0.2];
\node at (0,0)
{\pgfmathparse{TD("(R)")}%
$\vec R=(\ppv)$};
\end{tikzpicture}
```

The main usage of `\pgfmathprintvector` is to strip off unnecessary zeros, which emerge since the internal computations are largely done with the `fpu` library.

$$(1,0,0)^T \times (0,1,0)^T = (0,0,1)^T$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("(1,0,0)x(0,1,0)")}%
$(1,0,0)^T\times(0,1,0)^T=
(\pgfmathprintvector\pgfmathresult)^T$
```

$$\vec{A} \cdot \vec{B} = 5$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("(A)o(B)")}%
$\vec A\cdot \vec B=
\pgfmathprintnumber\pgfmathresult$
```

Notice that, as of now, the only purpose of brackets `(...)` is to delimit vectors. Further, the addition `+` and subtraction `-` have a *higher* precedence than vector products `x` and scalar products `o`. That is, `(A)+(B)o(C)` gets interpreted as $(\vec{A} + \vec{B}) \cdot \vec{C}$, and `(A)+(B)x(C)` as $(\vec{A} + \vec{B}) \times \vec{C}$.

$$(\vec{A} + \vec{B}) \cdot \vec{C} = -11$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("(A)+(B)o(C)")}%
$(\vec A+\vec B)\cdot\vec C=
\pgfmathprintnumber\pgfmathresult$
```

$$(\vec{A} + \vec{B}) \times \vec{C} = (9, -5, -1)$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("(A)+(B)x(C)")}%
$(\vec A+\vec B)\times\vec C=
(\pgfmathprintvector\pgfmathresult)$
```

You can use square brackets [...] to separately parse subexpressions with the standard PGF parser.

$$\sin(45) \vec{A} + \cos(30) \vec{B} = (2.44, 4.01, 1.26)^T$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{TD("{[\sin(45)]*(A)+[\cos(30)]*(B)}")}%
$\sin(45)\,\vec{A}+\cos(30)\,\vec{B}=
(\pgfmathprintvector\pgfmathresult)^T$
```

Moreover, any expression can only have either one `o` or one `x`, or none of these. Expressions with more of these can be accidentally right.

`axisangles("vector")`

Yields the the rotation angles that transforms the vector in the z -axis. Since an axis has a residual rotation symmetry, namely the rotation around this axis, only two angles are required, and thus returned. In the conventions of section 1.2, these are the angles ϕ and ψ . It corresponds to the macro `\tdplotgetpolarcoords{⟨x⟩{⟨y⟩}{⟨z⟩}}` from the `tikz-3dplot` package.

$$\angle(\vec{A}) = 63.43, 36.7$$

```
\usetikzlibrary {3dtools}
\pgfmathparse{axisangles("A")}%
$\sphericalangle(\vec{A})=
\pgfmathprintvector\pgfmathresult$
```

Computations in 3d sometimes involve projections of a point on a line or a plane. In 2d, projections of a point on a line can be conveniently done with the `calc` library, but this does in general not yield the correct projection in 3d. In order to make these projections more user friendly, `3dtools` offers the possibility to define extended objects such as lines or planes. This is the same concept as what is offered by plain TikZ, where the user can name coordinates and nodes, and, if the `intersections` library is loaded, also paths.

`/tikz/3d/plane through=point 1 and point 2 and point 3` named *name* (no default)

Defines a plane of name *name* by the requirement that it goes through the three specified points. Internally the plane is stored in terms of its normal and a point it goes through.

`/tikz/3d/plane with normal=normal` through *point 1* named *name* (no default)

Defines a plane of name *name* by the requirement that it goes through *point 1* and has the normal *normal*. Internally the plane is stored in terms of this normal and point.

`/tikz/3d/line through=point 1 and point 2` named *name* (no default)

Defines a line of name *name* by the requirement that it goes through *point 1* and *point 1*.

`/tikz/3d/line with direction=vector` through *point 1* named *name* (no default)

Defines a line of name *name* by the requirement that it goes through *point 1* and has direction *vector*.

`/tikz/3d/sphere with center=center` and radius *radius* named *name* (no default)

Defines a sphere with center *center* and radius *radius*.

`/tikz/3d/intersection of=object 1` with *object 2* (no default)

Computes the intersection of a named object *object 1* with another object *object 2*. Currently only intersections of lines with lines and planes with lines are supported. There is also some support for intersections of lines with spheres. The intersections get also named by some predefined names. Note that if intersections cannot be found, warnings will be issued. Due to the limited precision of $\text{\TeX}$ the decision whether or not an intersection actually exists may be wrong in close cases.

`/tikz/3d/aux keys/intersection 1` (initially I-1)

Predefined name for the first intersection.

`/tikz/3d/aux keys/intersection 2` (initially I-2)

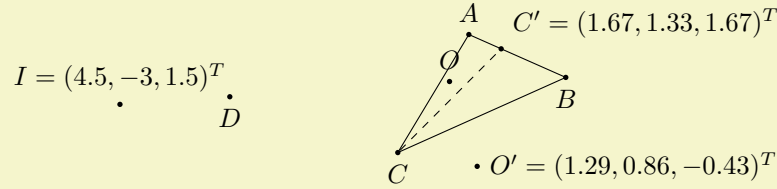
Predefined name for the second intersection.

These extended objects can be used in projections.

`/tikz/3d/project=point on named object` (no default)

Computes the projection of a point on some extended object, and inserts the projection in the path.

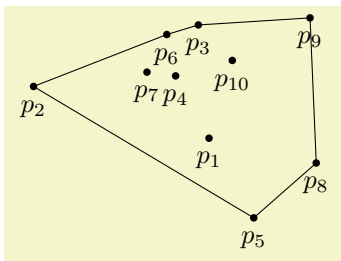
The following code example illustrates the usage. It also makes use of the `install view` key, which we describe in section 1.2.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,psi=0,theta=60},
dot/.style={circle,inner sep=0pt,
minimum size=2pt,fill}]
\draw[every coordinate node/.append style={dot}]
(2,1,2) coordinate[label=above:{$A$}] (A) --
(1,2,1) coordinate[label=below:{$B$}] (B) --
(2,0,0) coordinate[label=below:{$C$}] (C) -- cycle
(0,0,0) coordinate[label=above:{$O$}] (O)
(3,-2,1) coordinate[label=below:{$D$}] (D);
\path[3d/plane through={($A$) and ($B$) and ($C$) named $pABC$},
3d/plane with normal={($1,1,1$) through ($C$) named $pTwo$},
3d/line through={($A$) and ($B$) named $LAB$},
3d/line through={($O$) and ($D$) named $LOD$}];
% project point on plane
\path[3d/project={($O$) on $pABC$}] coordinate (O');
% project point on line
\path[3d/project={($C$) on $LAB$}] coordinate (C');
% intersection of plane and line
\path[3d/intersection of={$LOD$ with $pABC$}] coordinate (I);
\draw[dashed] (C) -- (C')
coordinate[dot,label=above right:{$C'$=\pgfmathparse{TD("$C'$")}\%
(\pgfmathprintvector{\pgfmathresult})^T$}];
\path (O') coordinate[dot,label=right:{$O'$=\pgfmathparse{TD("$O'$")}\%
(\pgfmathprintvector{\pgfmathresult})^T$}];
\path (I) coordinate[dot,label=above:{$I$=\pgfmathparse{TD("$I$")}\%
(\pgfmathprintvector{\pgfmathresult})^T$}];
\end{tikzpicture}
```

`/tikz/convex hull of={list of coordinates}` (no default)

Style that inserts a path for the 2-dimensional convex hull of a list of named coordinates.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}
\path foreach \X in {1,...,10}
{ (rnd*360:rnd*3) coordinate (p\X)
node[circle,inner sep=1pt,fill,label=below:{$p_{\X}$}]{}};
\draw[convex hull of={p1,p2,p3,p4,p5,p6,p7,p8,p9,p10}];
\end{tikzpicture}
```

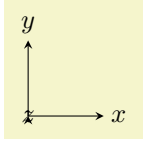
1.2 Orthonormal projections

The `3dtools` library can be used together with the `tikz-3dplot` package and/or the `perspective` library. It also has its own means to install orthonormal projections. Orthonormal projections emerge from subjecting 3-dimensional vectors to orthogonal transformations and projecting them to 2 dimensions. They are not to be confused with the perspective projections, which are more realistic and supported by the `perspective` library. Orthonormal projections may be thought of a limit of perspective projections at large distances, where large means that the distance of the observer is much larger than the dimensions of the objects that get depicted.

`/tikz/3d/install view` (no value)

Installs a 3d orthonormal projection.

The initial projection is such that x is right and y is up, as if we had no third direction.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view]
\draw[-stealth] (0,0,0) -- (1,0,0)
node[pos=1.2] {$x$};
\draw[-stealth] (0,0,0) -- (0,1,0)
node[pos=1.2] {$y$};
\draw[-stealth] (0,0,0) -- (0,0,1)
node[pos=1.2] {$z$};
\end{tikzpicture}
```

The 3d-like pictures emerge by rotating the view. The conventions for the parametrization of the orthogonal rotations in terms of three rotation angles ϕ , ψ and θ are

$$O(\phi, \psi, \theta) = \begin{pmatrix} c_\phi c_\psi & s_\phi c_\psi & -s_\psi \\ c_\phi s_\psi s_\theta - s_\phi c_\theta & s_\phi s_\psi s_\theta + c_\phi c_\theta & c_\psi s_\theta \\ c_\phi s_\psi c_\theta + s_\phi s_\theta & s_\phi s_\psi c_\theta - c_\phi s_\theta & c_\psi c_\theta \end{pmatrix}. \quad (1)$$

Here, $c_\phi := \cos \phi$, $s_\phi := \sin \phi$ and so on.

`/tikz/3d/phi` (initially 0)

3d rotation angle.

`/tikz/3d/psi` (initially 0)

3d rotation angle.

`/tikz/3d/theta` (initially 0)

3d rotation angle.

The rotation angles can be used to define the view. The conventions are chosen in such a way that they resemble those of the `tikz-3dplot` package, which gets widely used. This matrix can be written as

$$O(\phi, \psi, \theta) = R_x(\theta) \cdot R_y(\psi) \cdot R_z(\phi),$$

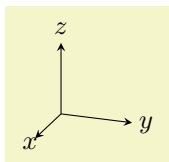
where

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) \\ 0 & -\sin(\theta) & \cos(\theta) \end{pmatrix}, \quad R_y(\psi) = \begin{pmatrix} \cos(\psi) & 0 & -\sin(\psi) \\ 0 & 1 & 0 \\ \sin(\psi) & 0 & \cos(\psi) \end{pmatrix},$$

$$R_z(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & \sin(\phi) \\ 0 & -\sin(\phi) & \cos(\phi) \end{pmatrix}$$

are rotations about the x , y and z axis, respectively. For $\psi = 0$, $O(\phi = \phi_d, \psi = 0, \theta = \theta_d) = R^d(\theta_d, \phi_d)$ from the `tikz-3dplot` package. Note, however, that there seems to be an inconsistency in equation (2.1) of that package.³

³I do not know how to contact the author.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,psi=0,theta=70}]
\draw[-stealth] (0,0,0) -- (1,0,0)
node[pos=1.2] {$x$};
\draw[-stealth] (0,0,0) -- (0,1,0)
node[pos=1.2] {$y$};
\draw[-stealth] (0,0,0) -- (0,0,1)
node[pos=1.2] {$z$};
\end{tikzpicture}
```

/tikz/3d/define orthonormal dreibein= $\langle options \rangle$ (no default, initially empty)

Defines a local dreibein from three input points. The initial choice of these points are $A=(A)$, $B=(B)$, $C=(C)$, and the initial choice for the basis vectors is $\mathbf{ex}=(\mathbf{ex})$, $\mathbf{ey}=(\mathbf{ey})$, $\mathbf{ez}=(\mathbf{ez})$. In terms of these points, $(\mathbf{ex})$ is parallel to $(B) - (A)$, $(\mathbf{ey})$ is in the plane that contains (A) , (B) and (C) , and is orthogonal to $(\mathbf{ex})$, and $(\mathbf{ez})$ is orthogonal to $(\mathbf{ex})$ and $(\mathbf{ey})$. All the basis vectors are normalized to length 1. To draw in the plane of the dreibein, use `3d/install local frame=from dreibein` (section 1.3). Using the basis vectors directly, as in $\mathbf{x}=\{\mathbf{ex}\}$, only works if they have been defined without a shift because $\mathbf{x}=\{\mathbf{ex}\}$ uses the position of $(\mathbf{ex})$ on the canvas.

/tikz/3d/screen coords (no value)

Defines locally screen coordinate system, i.e. this style brings you back to the original coordinate system in `tikzpictures`. It is taken from `tikz-3dplot`, where this style is called `tdplot_screen_coords`.

1.3 Main and local frames

The coordinates (x, y, z) that are used after a view has been installed, e.g. with `3d/install view`, refer to the *main* frame. Many objects are much easier to describe in a *local* (or secondary) frame. For instance, a circle in a tilted plane is just `(0,0) circle[radius=1]` in a frame whose xy plane is the tilted plane. A local frame is given by an origin $\vec{O}$ and three basis vectors $\vec{e}_x$, $\vec{e}_y$ and $\vec{e}_z$, all specified in main coordinates. The point with local coordinates (a, b, c) is

$$\vec{P} = \vec{O} + a\vec{e}_x + b\vec{e}_y + c\vec{e}_z$$

in the main frame.

/tikz/3d/install local frame= $\langle options \rangle$ (no default)

Installs a local frame. It sets the $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{z}$ vectors to the screen images of $\vec{e}_x$, $\vec{e}_y$ and $\vec{e}_z$, and shifts the origin to $\vec{O}$. Afterwards, $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ refers to the local frame, and so do $(\mathbf{a}, \mathbf{b})$, which TikZ reads as $(\mathbf{a}, \mathbf{b}, 0)$, polar coordinates and circles, ellipses and arcs with dimensionless radii. Things that live on the canvas, such as rectangles and node texts, are not affected; use `canvas is xy plane at z=0` (and `transform shape`) inside the local frame to put them in its xy plane. The vectors in $\langle options \rangle$ are 3d expressions in the sense of TD, i.e. they can be explicit, like $\{(1,0,0)\}$, named, like $(\mathbf{ex})$, or combinations, like $(B) - (A)$. Values that contain commas must be enclosed in braces. The vectors always refer to the main frame, even if another local frame is active; local frames do not compose. Like all TikZ transformations, the key takes effect after the other options of the scope or path it is given to. If it fails because the basis vectors are (almost) coplanar, a warning is issued and nothing changes.

/tikz/3d/local frame/origin= $\langle point \rangle$ (no default, initially $(0,0,0)$)

Origin of the local frame.

/tikz/3d/local frame/ex= $\langle vector \rangle$ (no default, initially $(1,0,0)$)

Local x basis vector.

/tikz/3d/local frame/ey= $\langle vector \rangle$ (no default, initially $(0,1,0)$)

Local y basis vector.

/tikz/3d/local frame/ez= $\langle vector \rangle$ (no default, initially empty)

Local z basis vector. If empty, $\vec{e}_z = \vec{e}_x \times \vec{e}_y$.

`/tikz/3d/local frame/orthonormalize=<boolean>` (default `true`, initially `false`)

If true, the basis is replaced by the right-handed orthonormal basis with $\vec{e}'_x \parallel \vec{e}_x$ and $\vec{e}'_y$ in the plane of $\vec{e}_x$ and $\vec{e}_y$; $\vec{e}_z$ is ignored. Otherwise the vectors are used as they are, so the local frame may be skewed or scaled.

`/tikz/3d/local frame/name=<name>` (no default, initially `empty`)

Saves the frame under $\langle name \rangle$ for later use with `3d/use local frame`.

`/tikz/3d/local frame/from dreibein` (no value)

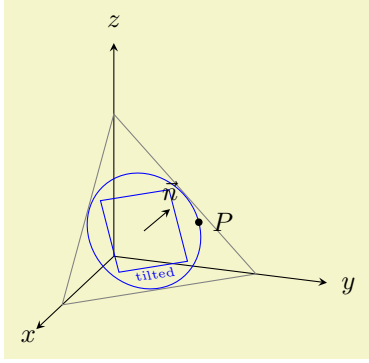
Uses the frame defined by the last `3d/define orthonormal dreibein`, i.e. origin A and basis vectors $\vec{e}_x$, $\vec{e}_y$ and $\vec{e}_z$.

`/tikz/3d/use local frame=<name>` (no default)

Installs a local frame that has been saved with `name=<name>`.

`/tikz/3d/main frame` (no value)

Goes back to the main frame inside a local frame, e.g. for a single path. The main frame consists of the x , y and z vectors and the transformation that were in force when the first local frame of the current scope got installed.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,psi=0,theta=70},
dot/.style={circle,inner sep=1pt,fill}]
\draw[-stealth] (0,0,0) -- (3,0,0) node[pos=1.1]{$x$};
\draw[-stealth] (0,0,0) -- (0,3,0) node[pos=1.1]{$y$};
\draw[-stealth] (0,0,0) -- (0,0,3) node[pos=1.1]{$z$};
\path (2,0,0) coordinate (A) (0,2,0) coordinate (B)
(0,0,2) coordinate (C) (2/3,2/3,2/3) coordinate (I);
\draw[gray] (A) -- (B) -- (C) -- cycle;
\begin{scope}[3d/install local frame={origin=(I),
ex=(B)-(A),ey=(C)-(A),orthonormalize,name=tilted}]
\draw[blue] (0,0) circle[radius=0.8];
\draw[blue] (-.5,-.5) -- (.5,-.5) -- (.5,.5)
-- (-.5,.5) -- cycle;
\draw[-stealth] (0,0,0) -- (0,0,1) node[above]{$\vec{n}$};
\begin{scope}[canvas is xy plane at z=0,transform shape]
\node[blue] at (0,-0.6) {\tiny tilted};
\end{scope}
\end{scope}
\path (0.8,0) coordinate[dot,label=right:$P$] (P);
\end{scope}
\end{tikzpicture}
```

Inside the picture, `TD("P")` yields the main coordinates of P outside the local frame and its local coordinates $(0.8, 0, 0)$ inside it (section 1.1). Outside the picture, as in the next example, it yields the coordinates P was defined with, i.e. the local ones. The following functions convert explicit coordinates between the frames. They use the local frame that is in force or, outside of any local frame, the one that was installed last. Results are accurate to about 10^{-4} .

`TDtomain("vector")`

Yields the main coordinates of a point given in local coordinates.

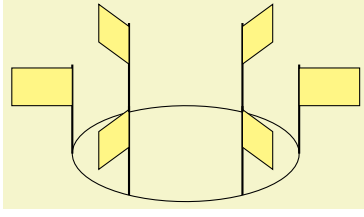
`TDtolocal("vector")`

Yields the local coordinates of a point given in main coordinates.

$$\begin{aligned}\vec{P}_{\text{main}} &= (0.1, 1.23, 0.67), \\ \vec{I}_{\text{loc}} &= (0, 0, 0), \\ \vec{A}_{\text{loc}} &= (-1.41, -0.82, 0)\end{aligned}$$

```
\usetikzlibrary {3dtools}
\pgfmathsetmacro{\Pmain}{TDtomain("P")}%
\pgfmathsetmacro{\Iloc}{TDtolocal("I")}%
\pgfmathsetmacro{\Aloc}{TDtolocal("A")}%
\def\ppv#1{(\pgfmathprintvector[/pgf/number format/fixed]{#1})}%
$\begin{array}{l}
\vec{P}_{\mathrm{main}}=\ppv{\Pmain},\\
\vec{I}_{\mathrm{loc}}=\ppv{\Iloc},\\
\vec{A}_{\mathrm{loc}}=\ppv{\Aloc}
\end{array}$
```

Pics work in local frames, too. Here a flag pic, which is drawn in its own xy plane, gets placed at six positions on a circle.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=120,psi=0,theta=65},
pics/flag/.style={code={\draw[thick] (0,0) -- (0,1.3);
\draw[fill=yellow!60] (0,0.7) -- (0.8,0.7) -- (0.8,1.25)
-- (0,1.25) -- cycle;}}]
\draw (0,0,0) circle[radius=1.5];
\foreach \a in {0,60,...,300}
{\begin{scope}[3d/install local frame={
origin={{(1.5*cos(\a)),(1.5*sin(\a)),0}},
ex={{(cos(\a)),(sin(\a)),0}},ey={{(0,0,1)}}}
\pic{flag};
\end{scope}}
\end{tikzpicture}
```

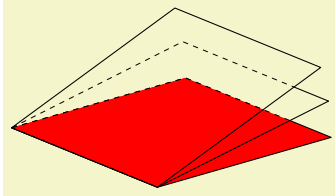
1.4 Visibility considerations

When drawing objects in 3d, some parts are “visible”, i.e. in the foreground, while other parts are “hidden”, i.e. in the background. In what follows, we discuss some basic means that allow one to determine whether some stretch is visible or hidden. Very often this amounts to find critical values of some parameter(s) (such as angles) at which some curve changes from visible to hidden, or vice versa. One of the major players in this game is the normal on the screen, which is given by the last row of the orthogonal matrix O of (1). For $\psi = 0$, this normal is also the normal in the so-called “main” coordinates of **tikz-3dplot** with the identifications $\theta = \texttt{\tdplotmaintheta}$ and $\phi = \texttt{\tdplotmainphi}$.

/tikz/3d/draw ordered paths= $\langle list \rangle$

(no default, initially empty)

This draws a list of named paths that have been created in with the **save named path** in the order they appear in the list $\langle list \rangle$. This is achieved by a number of clips and inverted clips. Notice that, at this point, layers are not supported. This is because clips have to be applied in the same layer as the path that is to be clipped against. One can define styles that carry the same names as the named paths in the directory **3d/ordered paths/**. If such a style exists, it will be applied.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=50,theta=70},
declare function={a=3;alpha=20;},
line join=round]
\path
(-a/2,0,0) coordinate (P_1)
(a/2,0,0) coordinate (P_2)
(-a/2,a,0) coordinate (A_1)
(a/2,a,0) coordinate (A_2)
(-a/2,{a*cos(alpha/2)},{a*sin(alpha/2)}) coordinate (B_1)
(a/2,{a*cos(alpha/2)},{a*sin(alpha/2)}) coordinate (B_2)
(-a/2,{a*cos(alpha)},{a*sin(alpha)}) coordinate (C_1)
(a/2,{a*cos(alpha)},{a*sin(alpha)}) coordinate (C_2);
\foreach \X in {A,B,C}
{\path[save named path=p\X] (P_1) -- (P_2) --
(\X_2) -- (\X_1) -- cycle;}
\pgfmathsetmacro{\nA}{TDunit("(A_1)-(P_1)x(P_2)-(P_1)")}
\pgfmathsetmacro{\nC}{TDunit("(C_1)-(P_1)x(P_1)-(P_2)")}
\pgfmathtruncatemacro{\itest}{screendepth(\nA)>screendepth(\nC)?1:0}
\tikzset{3d/ordered paths/pA/.style={fill=red}}
\ifnum\itest=1
\tikzset{3d/draw ordered paths={pC,pB,pA}}
\else
\tikzset{3d/draw ordered paths={pA,pB,pC}}
\fi
\end{tikzpicture}
```

1.4.1 Example 1: latitude circle on sphere

Here is an example. Suppose we wish to draw a latitude circle on a sphere for $\psi = 0$. Since the system has a rotational symmetry around the z axis, we can set $\phi = 0$ as well. The normal on the screen then

becomes

$$\vec{n}_{\text{screen}} = (0, -\sin(\theta), \cos(\theta))^T.$$

The latitude circle can be parametrized as

$$\vec{\gamma}_{\text{lat}}(\beta) = (r \cos(\beta), r \sin(\beta), h)^T,$$

where $r = R \cos(\lambda)$ and $h = R \sin(\lambda)$ with R being the radius of the sphere and λ the latitude angle of the latitude circle. The critical angles β_{crit} at which the curve transitions from hidden to visible are then given by the solutions of

$$\vec{n}_{\text{screen}} \cdot \vec{\gamma}_{\text{lat}}(\beta_{\text{crit}}) = -r \sin(\beta) \sin(\theta) + h \cos(\theta) \stackrel{!}{=} 0.$$

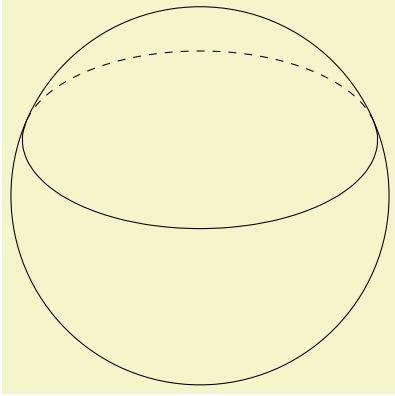
Since $-1 \leq \sin(\beta) \leq 1$, this equation only has solutions if

$$\left| \frac{h}{r} \cot(\theta) \right| \leq 1. \quad (2)$$

The solutions are then given by

$$\beta_{\text{crit}}^{(1)} = \arcsin\left(\frac{h}{r} \cot(\theta)\right) \quad \text{and} \quad \beta_{\text{crit}}^{(2)} = 180 - \beta_{\text{crit}}^{(1)},$$

and $\beta_{\text{crit}}^{(1)} = \beta_{\text{crit}}^{(2)}$ if the inequality (2) is saturated. The following code example illustrates this. Notice that it could be made more concise with the 3d library.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[declare function={%
R=2.5;lambda=20;theta=60;
r=R*cos(lambda);h=R*sin(lambda);
betacrit=asin(h/r*cot(theta)};]
\draw (0,0) circle[radius=R];
\begin{scope}[smooth,
3d/install view={phi=0,psi=0,theta=theta}]
\draw[dashed]
plot[variable=\t,
domain=betacrit:180-betacrit]
({r*cos(\t)},{r*sin(\t)},h);
\draw
plot[variable=\t,
domain=betacrit:-180-betacrit]
({r*cos(\t)},{r*sin(\t)},h);
\end{scope}
\end{tikzpicture}
```

1.4.2 Example 2: arbitrary circle on sphere

A somewhat more advanced question is to draw an arbitrary circle on a sphere. That is, we are given a sphere around a center C or radius R and a point P inside the sphere. If C and P coincide, we need to define a normal n , otherwise $n = P - C$. How can one draw a circle on the sphere, distinguishing between visible and hidden stretches?

First test: $|P - C| < R$. The circle only makes sense if $|P - C| < R$. The radius of the circle is $r = \sqrt{R^2 - |P - C|^2}$. Call the normal on the screen n_{screen} .

Case $n \parallel n_{\text{screen}}$. If n and n_{screen} are linearly dependent, the circle is completely hidden if the screen depth of P is smaller than the screen depth of C , $\text{sd } P < \text{sd } C$, and visible otherwise. We can choose a coordinate system in which

$$e_z = \frac{n}{|n|}, \quad e_x \text{ arbitrary but } \perp e_z \quad \text{and} \quad e_y = e_z \times e_x$$

to draw the circle.

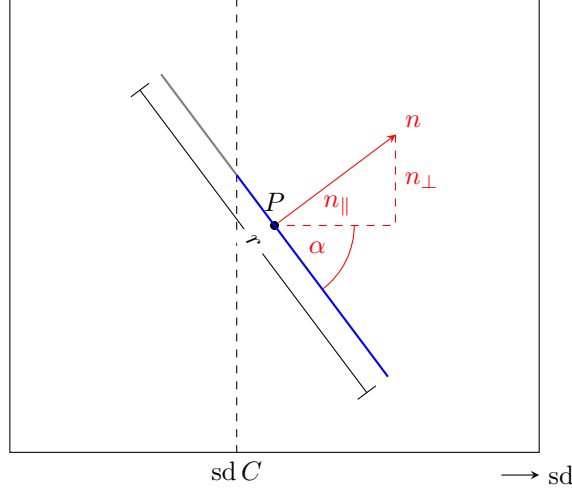


Figure 1: $n \not\parallel n_{\text{screen}}$.

Case $n \not\parallel n_{\text{screen}}$. From now on we assume that n and n_{screen} are not linearly dependent. This means that they span a plane, and we can choose n_{screen} to point in the x -direction of this plane (see figure 1).

The normal can be decomposed in a part parallel to n_{screen} , $n_{\parallel}$, and a perpendicular part, $n_{\perp}$. The projection of the circle on this plane is a line, and has slope $\alpha = \arctan(-n_{\parallel}/n_{\perp})$. If $r \cos \alpha > |\text{sd } P - \text{sd } C|$, the circle has both hidden and visible stretches, from between $-r \cos \alpha$ and $\text{sd } P - \text{sd } C$ it is hidden and between $\text{sd } P - \text{sd } C$ and $r \cos \alpha$ it is solid.

We are going to draw the circle in a coordinate system defined by

$$e_z = \frac{n}{|n|}, \quad e_y = e_z \times n_{\text{screen}} \quad \text{and} \quad e_x = e_y \times e_z.$$

The circle is then in the x - y plane, and the positive x -direction is the direction of increasing screen depth (or visibility). If $r \cos \alpha > |\text{sd } P|$, then the stretch between $\beta := |\arccos(\frac{\text{sd } C - \text{sd } P}{r \cos \alpha})|$ and $-\beta$ is visible, between β and $360^\circ - \beta$ it is hidden.

1.4.3 Example 3: outlines of shapes

Another type of visibility considerations concerns the contours of shapes. To be specific, consider a surface parametrized by

$$\vec{f}(u, v) = \begin{pmatrix} u \\ R(u) \cos(v) \\ R(u) \sin(v) \end{pmatrix} \quad (3)$$

with some function $R(u)$. What is the contour of an orthonormal projection of the surface on the screen? It is the boundary between the hidden and visible parts. What distinguishes the visible from the hidden parts? It is the projection of the normal of the surface on the screen, $F(u, v) := n_S(u, v) \cdot n_{\text{screen}}$. The hidden and visible stretches are separated by the zeros of this projection. In the case of our surface (3),

$$\vec{n}_S(u, v) = \vec{\nabla}_u f(u, v) \times \vec{\nabla}_v f(u, v) = R(u) \begin{pmatrix} -R'(u) \\ \cos(v) \\ \sin(v) \end{pmatrix}, \quad (4)$$

and for $\psi = 0$

$$\vec{n}_{\text{screen}} = \begin{pmatrix} \sin(\theta) \sin(\phi) \\ -\sin(\theta) \cos(\phi) \\ \cos(\theta) \end{pmatrix}. \quad (5)$$

So we need to find the zeros of

$$\begin{aligned} F(u, v) &= -\cos(\theta) \sin(v) + \sin(\theta) \cos(\phi) \cos(v) + R'(u) \sin(\theta) \sin(\phi) \\ &=: a \sin(v) + b \cos(v) + c. \end{aligned} \quad (6)$$

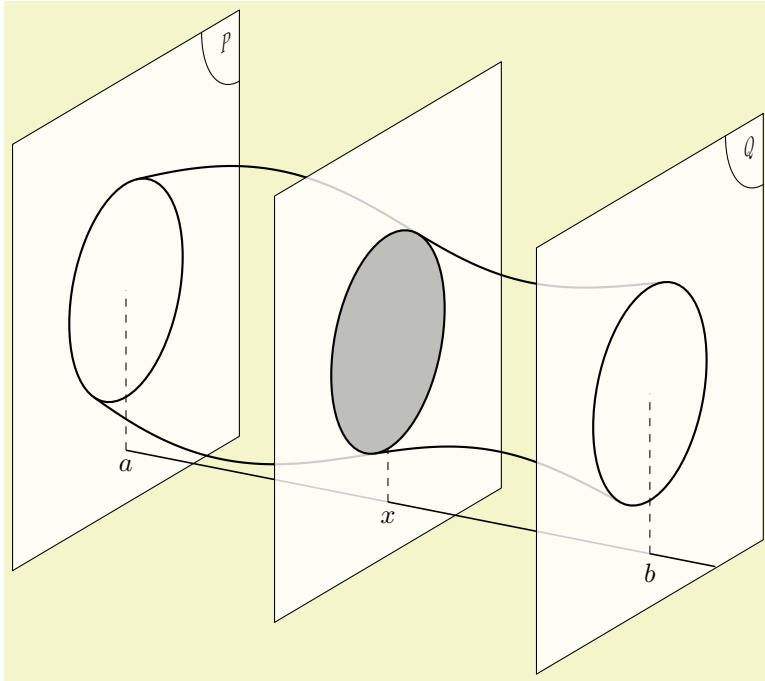
This equation can be solved for $x := \sin(v)$ using $\cos(v) = \pm\sqrt{1 - \sin^2(v)} = \pm\sqrt{1 - x^2}$. It is clear that, since $\sin(v)$ is bounded (and so is $\cos(v)$), this equation does not always have a solution. In particular, if $R'(u)$ is too large, there might not be a solution. This just means that for a given u all points are hidden or visible. This happens e.g. when you look at a cone from the top or bottom. A more advanced code could deal with these cases, the one presented below (which is taken from topanswers.xyz) does not. Note also that these are local visibility conditions, a seemingly visible stretch can always be covered by some other stretch that is a finite distance away in u direction.

However, for “reasonable” configurations, there are two solutions of the quadratic equation, as they should, they correspond to the upper and lower contour in the code example below. They are a bit unilluminating and given by

$$[\sin(v)]_{1,2} = -\frac{a\sqrt{a^2 + b^2 - c^2} \pm bc}{a^2 + b^2}, \quad (7a)$$

$$[\cos(v)]_{1,2} = -\frac{b\sqrt{a^2 + b^2 - c^2} \mp ac}{a^2 + b^2}, \quad (7b)$$

where a , b and c are implicitly defined in (6). Hence, for “reasonable” configurations, it is straightforward to plot the contour of the surface with $\text{\LaTeX}$.



```

\usetikzlibrary {3dtools}
\begin{tikzpicture}[declare function={phi=30;theta=70;},
3d/install view={phi=phi,psi=0,theta=theta},
line cap=butt,line join=round,
declare function={%
  R(\u)=1.5+sin(\u*45)/(2+\u/8);%<- input
  Rprime(\u)=(R(\u+0.01)-R(\u-0.01))/0.02;%<- numerical derivative
  a=cos(phi)*sin(theta);% see equation {eq:vcrit}
  b=-1*cos(theta);%
  c(\u)=sin(theta)*sin(phi)*Rprime(\u);%
  sv1(\u)=-((b*c(\u)+sqrt(a*a+b*b-c(\u)*c(\u))*abs(a))/(a*a+b*b));%
  sv2(\u)=-((b*c(\u)+sqrt(a*a+b*b-c(\u)*c(\u))*abs(a))/(a*a+b*b));%
  cv1(\u)=-((a*c(\u)+sqrt(a*a+b*b-c(\u)*c(\u))*abs(b))/(a*a+b*b));%
  cv2(\u)=-((a*c(\u)+sqrt(a*a+b*b-c(\u)*c(\u))*abs(b))/(a*a+b*b));%
},
icircle/.style={thick},
iplane/.style={fill=white,fill opacity=0.8},
iline/.style={semithick},
extra/.code={},annot/.code={\tikzset{extra/.code={#1}}}
\newcommand\DrawIntersectingPlane[2][]{%
\begin{scope}[canvas is yz plane at x=#2,transform shape,#1]
\draw[iplane] (3,3) -- (-3,3) -- (-3,-3) -- (3,-3) -- cycle;
\draw[icircle] (0,0) circle[radius={R(#2)}];
\tikzset{extra}
\end{scope}}
%
\DrawIntersectingPlane[annot={%
\path (2.9,2.9) node[below left,transform shape]{$P$};
\draw (2,3) arc[start angle=180,end angle=270,radius=1];
\draw[dashed] (0,-2.25) node[below,transform shape=false]{$a$} -- (0,0);
}] {0}
\draw[iline] (0,0,-2.25) -- (4,0,-2.25);
\draw[thick,smooth,variable=\u,domain=0:4]
plot (\u,{R(\u)*cv1(\u)},{R(\u)*sv1(\u)})
plot (\u,{R(\u)*cv2(\u)},{R(\u)*sv2(\u)});
\DrawIntersectingPlane[icircle/.append style={fill=gray,fill opacity=0.5},
annot={%
\draw[dashed] (0,-2.25) node[below,transform shape=false]{$x$} -- (0,-1.5);
}] {4}
\draw[iline] (4,0,-2.25) -- (8,0,-2.25);
\draw[thick,smooth,variable=\u,domain=4:8]
plot (\u,{R(\u)*cv1(\u)},{R(\u)*sv1(\u)})
plot (\u,{R(\u)*cv2(\u)},{R(\u)*sv2(\u)});
%
\DrawIntersectingPlane[annot={%
\path (2.9,2.9) node[below left,transform shape]{$Q$};
\draw (2,3) arc[start angle=180,end angle=270,radius=1];
\draw[dashed] (0,-2.25) node[below,transform shape=false]{$b$} -- (0,0);
}] {8}
\draw[iline] (8,0,-2.25) -- (9,0,-2.25);
\end{tikzpicture}

```

There are some special cases that may be of particular interest: $R'(u) = 0$ for a cylinder and $R'(u) = -r/h$ for a cone of height h and base radius r .

1.5 Automatic occlusion

TikZ paints in the order in which paths are given. Inside a scene, the library reorders them instead: every path becomes a *piece*, its output is put aside while TikZ builds it, and at the end of the scene the pieces are painted from back to front (painter's algorithm). The depth of a piece is the mean depth of the points the path visits (section 1.1). The pics `cylinder`, `3d/frustum`, `3d/cone`, `3d/shaded cone`, `3d/circle` and `3d/circle on sphere` split themselves into pieces (the rim behind, the front half of the mantle, the cap, the arcs in front and behind), each with the depth of its centroid, so that, e.g., a ring around a cylinder passes behind it and in front of it. Faces of polyhedra are paths, so they are pieces, too.

A straight path whose depth varies (an axis, a beam) is cut where it passes other pieces (at their depths), and each run is painted at its own depth, so it can pass through them (key `3d/split`). Pieces of equal depth, like the coplanar caps of nested cylinders, are painted larger first. A `3d/sphere` places the pieces around it: what is inside stays between its back and front shells, what lies on its surface (points,

the arcs of `3d/circle on sphere`) goes just in front of the shell it lies on, in the order of the code on the front and in the reverse order on the back (what is on top in front is at the bottom behind), and what is outside goes behind or in front of the whole sphere. A straight path is also cut where it passes the surface of a sphere. In a scene the hidden edges of the pics are drawn like visible ones: an opaque surface covers them, a translucent one tints them.

This is a heuristic, not hidden surface removal. Each piece gets a single depth, so curved pieces that interpenetrate may get the wrong order; set the depth by hand with `3d/depth` where needed. A circle drawn with the `circle` path operation is a single piece at the depth of its center; use the pic `3d/circle` for rings that wrap around other objects, and `3d/sphere` for a sphere with things inside. Translucent fills keep working.

`/tikz/3d/scene` (no value)

Turns the current scope (or picture) into a scene. Nested scenes belong to the outermost one. A piece goes to the layer that was active when the path started. The line width, dash pattern, colors, opacity, line cap and join set by scopes inside the scene are restored in every piece; clips and transparency groups of such scopes are not, and should be put on the paths (or around the scene). Paths that visit no point with a known depth, like a lone node at a named coordinate, are painted in front.

`/tikz/3d/split=<length>` (no default, initially 0.2mm)

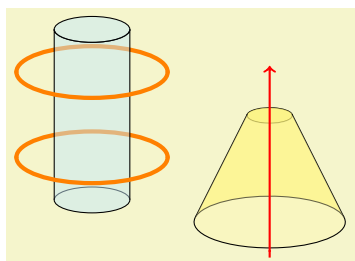
A path whose points lie on a straight line, that does not fill, and whose depth varies by more than $\langle length \rangle$ is cut where it passes other pieces (the runs are clipped copies of the same drawing, overlapping by 0.25pt). 0 switches this off.

`/tikz/3d/edge only` (no value)

Removes the fill, the shading and the path picture of a path. The pics add it to `3d/hidden`, so the style of a surface can be reused for its hidden edges.

`/tikz/3d/depth=<depth>` (no default)

Sets the depth of the pieces of the current path or scope. $\langle depth \rangle$ is `front`, `back`, or a coordinate in parentheses whose depth is used, e.g. `3d/depth={(0,0,1)}`.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=20,theta=70},
  3d/scene]
% the rings come first in the code
\foreach \z in {0.6,1.8}
\pic[orange,ultra thick] at (0,0,\z) {3d/circle={r=1}};
\pic {cylinder={r=0.5,h=2.4,
  cylinder/mantle/.style={fill=cyan!20,fill opacity=0.5,draw},
  cylinder/top/.style={fill=cyan!20,fill opacity=0.5,draw}}};
\pic at (2.5,0,0) {3d/frustum={R=1,r=0.3,h=1.5,
  frustum/inner/.style={fill=yellow!70,fill opacity=0.7},
  frustum/outer/.style={fill=yellow!40,fill opacity=0.4}}};
\draw[red,thick,->] (2.5,0,-0.5) -- (2.5,0,2.2);
\end{tikzpicture}
```

1.6 “Physical” coordinates

It is quite conceivable that users may want to use different coordinate systems for defining different coordinates. The following functions aim at supporting this. It should be said, though, that these functions are even more “experimental” than what has been discussed thus far. The main point is that, apart from the “physical” x and y coordinates, i.e. the coordinates of a point on the screen, one can also keep track of the physical z coordinate, or screen depth, which indicates how far a given point is above ($z > 0$) or below ($z < 0$) the screen. The physical components of every named node and coordinate are recorded (section 1.1).

`/tikz/3d/record physical components` (no value)

Does nothing; kept for compatibility. Earlier versions needed it to switch on the recording.

The physical components of a point are its x and y coordinates on the canvas, in **cm**, and its screen depth $\vec{n} \cdot \vec{P}$, where $\vec{P}$ are the main coordinates of the point (section 1.3) and $\vec{n}$ is the normal on the screen of the main frame (cf. `nscreenx` etc.). For views installed with `3d/install view` this means that the physical components are $O(\phi, \psi, \theta) \cdot \vec{P}$ of (1), plus the canvas shift of the main frame. The rules for the depth are those of section 1.1; if the depth of a coordinate is not known, `TDphysz` warns and yields 0. The physical components can then be retrieved with various functions.

`TDphysx("vector")`

Yields the physical x -component of a 3d named coordinate.

`TDphysy("vector")`

Yields the physical y -component of a 3d named coordinate.

`TDphysz("vector")`

Yields the physical z -component of a 3d named coordinate.

`TDphys("vector")`

Yields an array containing physical x , y and z -components of a 3d named coordinate.

Sometimes one may want to retrieve the components of a physical coordinate in another frame, i.e. the coordinates of a point that has been defined in one frame with respect to the frame that is in force now. These functions invert the map from coordinates to physical components exactly, so they work for any non-degenerate view and local frame. Shifts are taken into account; other transformations, e.g. `scale`, should be the same as when the point was recorded.

`TDlocalx("vector")`

Yields the local x -component of a 3d named coordinate the physical coordinates of which have been recorded.

`TDlocaly("vector")`

Yields the local y -component of a 3d named coordinate the physical coordinates of which have been recorded.

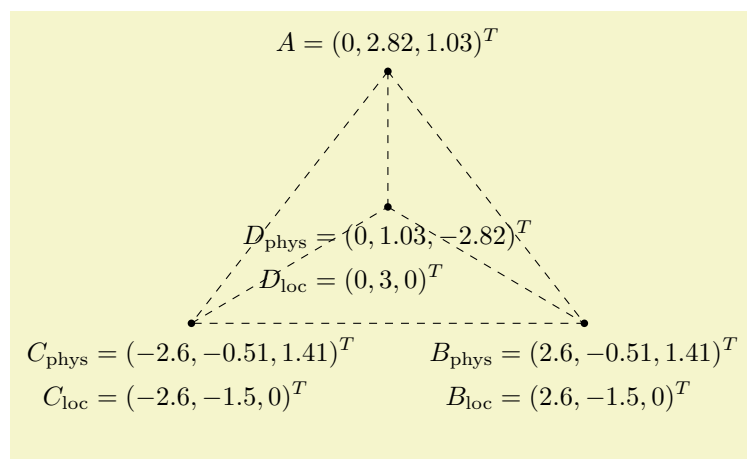
`TDlocalz("vector")`

Yields the local z -component of a 3d named coordinate the physical coordinates of which have been recorded.

`TDlocal("vector")`

Yields an array containing local x , y and z -components of a 3d named coordinate the physical coordinates of which have been recorded.

In the following example, B , C and D are defined at $(3,0,0)$ in views that are rotated by $\phi = 30^\circ$, 150° and 270° , respectively. `TDlocal`, evaluated in the unrotated view, yields their coordinates in this view.



```

\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/record physical components,
dot/.style={circle,inner sep=1pt,fill}]
\begin{scope}[3d/install view={phi=0,psi=0,theta=70}]
\path (0,0,3) coordinate (A);
\path[3d/install view={phi=30}] (3,0,0) coordinate (B);
\path[3d/install view={phi=150}] (3,0,0) coordinate (C);
\path[3d/install view={phi=270}] (3,0,0) coordinate (D);
\draw[dashed] foreach \X [remember=\X as \Y (initially D)]
in {B,C,D}
{(A) -- (\X) (\Y) -- (\X)}
coordinate[dot,label=below:{$\begin{aligned}
\X_\mathrm{phys}&=\pgfmathparse{TDphys("\X")}%
(\pgfmathprintvector[/pgf/number format/fixed]\pgfmathresult)^T\|
\X_\mathrm{loc}&=\pgfmathparse{TDlocal("\X")}%
(\pgfmathprintvector[/pgf/number format/fixed]\pgfmathresult)^T\|
\end{aligned}$}];
(A) coordinate[dot,label=above:{$A=\pgfmathparse{TDphys("A")}%
(\pgfmathprintvector\pgfmathresult)^T
$}];
\end{scope}
\end{tikzpicture}

```

Once one has defined physical coordinates, one can use them to compute e.g. distances between coordinates defined in different frames.

$$\begin{aligned}
\vec{A}_{\text{phys}} &= (0, 2.82, 1.03), \\
\vec{B}_{\text{phys}} &= (2.6, -0.51, 1.41), \\
d_{\text{phys}}(\vec{A}, \vec{B}) &= 4.24
\end{aligned}$$

```

\usetikzlibrary {3dtools}
$\pgfmathsetmacro{\vecA}{TDphys("A")}%
\pgfmathsetmacro{\vecB}{TDphys("B")}%
\pgfmathsetmacro{\physdist}{%
sqrt(TD("(\vecA)-(\vecB)o(\vecA)-(\vecB)")}%
\begin{array}{l}
\vec A_\mathrm{phys}=(\pgfmathprintvector\vecA),\\
\vec B_\mathrm{phys}=(\pgfmathprintvector\vecB),\\
d_\mathrm{phys}(\vec A,\vec B)=%
\pgfmathprintnumber\physdist
\end{array}$

```

/tikz/3d/install dual basis

(no value)

Defines the coordinates (dualex), (dualexy) and (dualez) (the names are stored in the keys /tikz/3d/dual ex etc.) such that the local components are the scalar products of these vectors with the physical components measured from the origin of the frame in force. For orthonormal views and frames, they are the physical components of the basis vectors.

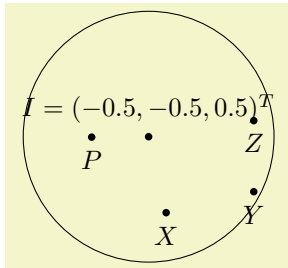
Section 1.1 lists the constructions through which the depth cannot be tracked, e.g. canvas is xy plane at z=0 (use a local frame instead).

1.7 Predefined path constructions and pics

/tikz/3d/circumsphere center=<options>

(no default, initially empty)

Computes the center of a sphere for a given set of four non-coplanar points. The initial choice of these points are A=(A), B=(B), C=(C), D=(D). The underlying maths can be found at <https://topanswers.xyz/tex?q=1233#a1467>.



```

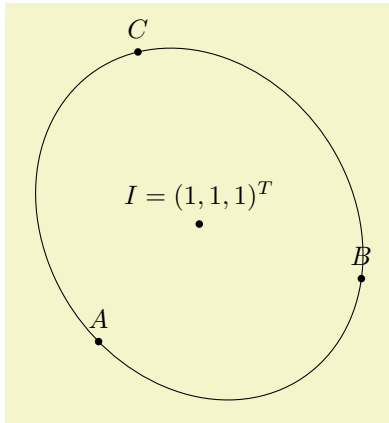
\usetikzlibrary {3dtools}
\begin{tikzpicture}[dot/.style={circle,inner sep=1pt,fill}]
\begin{scope}[3d/install view={phi=100,psi=0,theta=70}]
\path (1,0,0) coordinate (X)
(0,1,0) coordinate (Y)
(0,1,1) coordinate (Z)
(1,-1,1) coordinate (P);
\path[3d/circumsphere center={A={X},B={Y},C={Z},D={P}}]
coordinate (I);
\end{scope}
\pgfmathsetmacro{\csr}{sqrt(TD("(X)-(I)o(X)-(I)"))}
\draw (I) circle[radius=\csr];
\path foreach \X in {X,Y,Z,P}
{(\X) coordinate[dot,label=below:{$\X$}]}
(I) (I) coordinate[dot,label=above:{$I=\pgfmathparse{TD("(I)")}%
(\pgfmathprintvector\pgfmathresult)^T$}];
\end{tikzpicture}

```

`/tikz/3d/circumcircle center=<options>`

(no default, initially empty)

Computes the center of a circle for a given set of three non-collinear points. The initial choice of these points are A=(A), B=(B), C=(C).



```

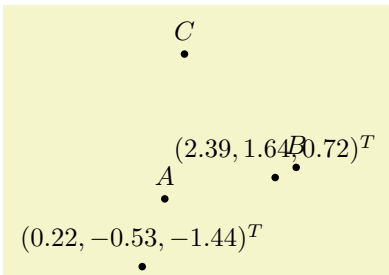
\usetikzlibrary {3dtools}
\begin{tikzpicture}[declare function={a=3;},
dot/.style={circle,inner sep=1pt,fill},
3d/install view={phi=100,psi=0,theta=70}]
\path (a,0,0) coordinate (A)
(0,a,0) coordinate (B)
(0,0,a) coordinate (C);
\path[3d/circumcircle center]
coordinate (I);
\pgfmathsetmacro{\myr}{sqrt(TD("(A)-(I)o(A)-(I)"))}
\tikzset{3d/define orthonormal dreibein}
\begin{scope}[x={\ex},y={\ey}]
\draw (I) circle[radius=\myr];
\end{scope}
\path foreach \X in {A,B,C}
{(\X) coordinate[dot,label=above:{$\X$}]}
(I) (I) coordinate[dot,label=above:{$I=\pgfmathparse{TD("(I)")}%
(\pgfmathprintvector\pgfmathresult)^T$}];
\end{tikzpicture}

```

`/tikz/3d/intersection of three spheres=<options>`

(no default, initially empty)

Computes the intersection intersections of three spheres around A=(A), B=(B), and C=(C). The radii are stored in rA, rB and rC, respectively. The names of the intersection solutions are stored in the keys i1 and i2, the initial values of which are i1 and i2.



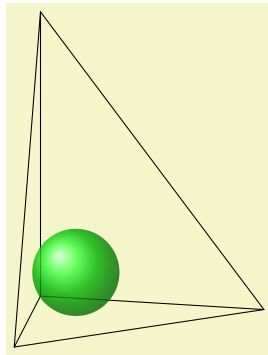
```

\usetikzlibrary {3dtools}
\begin{tikzpicture}[declare function={a=1.5;},
dot/.style={circle,inner sep=1pt,fill},
3d/install view={phi=100,psi=0,theta=70}]
\path (a,0,0) coordinate (A)
(0,a,0) coordinate (B)
(0,0,a) coordinate (C);
\path[3d/intersection of three spheres=
{rA=2,rB=2.5,rC=3}];
\path (i1)
coordinate[dot,label=above:{$%
\pgfmathparse{TD("(i1)")}%
(\pgfmathprintvector\pgfmathresult)^T$}];
\path (i2)
coordinate[dot,label=above:{$%
\pgfmathparse{TD("(i2)")}%
(\pgfmathprintvector\pgfmathresult)^T$}];
\path foreach \X in {A,B,C}
{(\X) coordinate[dot,label=above:{$\X$}]};
\end{tikzpicture}

```

/tikz/3d/insphere center= $\langle options \rangle$ (no default, initially empty)

Computes the center of an insphere center of a tetrahedron defined by four corners, A, B, C, and D. The initial values are A=(A), B=(B), C=(C), and D=(D).



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[line join=round,line cap=round,
3d/install view={phi=100,psi=0,theta=70}]
\path (0,0,0) coordinate (D)
(2,0,0) coordinate (A)
(0,3,0) coordinate (B)
(0,0,4) coordinate (C);
\path[3d/insphere center] coordinate (I);
\tikzset{3d/plane through=(A) and (B) and (C) named pABC}
\path[3d/project={ (I) on pABC}] coordinate (I');
\pgfmathsetmacro{\myr}{tddistance("(I)","(I')")}
\draw (A) -- (B) -- (C) -- cycle --(D) --(B) (C) -- (D);
\shade[3d/screen coords,ball color=green, opacity=0.8]
(I) circle [radius=\myr];
\end{tikzpicture}
```

/tikz/pics/3d circle through 3 points= $\langle options \rangle$ (no default, initially empty)

Draws a circle through 3 points in 3 dimensions. If the three coordinates are close to linearly dependent, the circle will not be drawn. *This pic will most likely be removed from the documentation and no longer be supported/developed.*

/tikz/3d circle through 3 points/A (initially (1,0,0))

First coordinate. Can be either symbolic or explicit. Symbolic coordinates need to be defined via `\path (x,y,z) coordinate (name);`.

/tikz/3d circle through 3 points/B (initially (0,1,0))

Second coordinate, like above.

/tikz/3d circle through 3 points/C (initially (0,0,1))

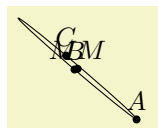
Third coordinate, like above.

/tikz/3d circle through 3 points/center name (initially M)

Name of the center coordinate that will be derived.

/tikz/3d circle through 3 points/auxiliary coordinate prefix (initially tmp)

In TikZ the coordinates are global. The code for the circle is more comprehensible if named coordinates are introduced. Their names will begin with this prefix. Changing the prefix will allow users to avoid overwriting existing coordinates.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={%
phi=30,psi=0,theta=70}]
\foreach \X in {A,B,C}
{\pgfmathsetmacro{\myx}{3*(rnd-1/2)}
\pgfmathsetmacro{\myy}{3*(rnd-1/2)}
\pgfmathsetmacro{\myz}{3*(rnd-1/2)}
\path (\myx,\myy,\myz) coordinate (\X);}
\path pic{3d circle through 3 points={%
A={ (A) },B={ (B) },C={ (C) },center name=MM}};
\foreach \X in {A,B,C,MM}
{\fill (\X) circle[radius=1.5pt]
node[above]{$\X$};}
\end{tikzpicture}
```

/tikz/pics/3d/circle on sphere= $\langle options \rangle$ (no default, initially empty)

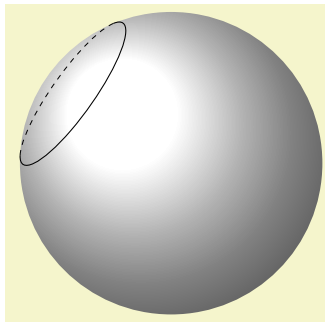
Draws a circle (or an arc) on a sphere. It distinguishes between foreground and background. Notice that this pic defines its own dreibein, so it will define coordinates that draw their names from `/tikz/3d/aux keys/ex`, `/tikz/3d/aux keys/ey` and `/tikz/3d/aux keys/ez`. It will also define

a coordinate (`nscreen`) which contains the normal to the screen. The foreground path will use the style `/tikz/3d/visible`, and the hidden path will be drawn with the style `/tikz/3d/hidden`. The commands of the `tikz-3dplot-circleofsphere` package are available through the library `3dtools.circleofsphere` (section 1.12).

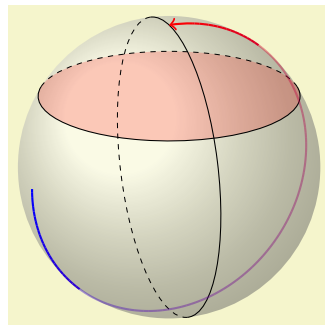
- `/tikz/3d/circle on sphere/C` (initially (0,0,0))
Coordinate of the center of the sphere.
- `/tikz/3d/circle on sphere/P` (initially (0,0,0))
Coordinate of the center of the circle.
- `/tikz/3d/circle on sphere/n` (initially (0,0,1))
Normal. This is only needed if the center of the circle coincides with the center of the sphere.
- `/tikz/3d/circle on sphere/auxiliary coordinate prefix` (initially `tmp`)
This pic installs a symbolic coordinate for the center of the circle. The auxiliary prefix can be used to avoid overwriting existing coordinates. The initial choice of the prefix will lead to a coordinate with name (`tmpP`).
- `/tikz/3d/circle on sphere/fore layer= $\langle layer \rangle$` (no default, initially `main`)
Layer of the visible faces.
- `/tikz/3d/circle on sphere/back layer= $\langle layer \rangle$` (no default, initially `main`)
Layer of the hidden faces.
- `/tikz/3d/circle on sphere/latitude= $\langle angle \rangle$` (no default, initially `empty`)
The latitude circle at $\langle angle \rangle$: sets `n` to (0,0,1) and `P` to `C` shifted by $R \sin(\langle angle \rangle)$ along z .
- `/tikz/3d/circle on sphere/longitude= $\langle angle \rangle$` (no default, initially `empty`)
The meridian at the azimuth $\langle angle \rangle$: the great circle through the poles and the direction $(\cos \langle angle \rangle, \sin \langle angle \rangle, 0)$.
- `/tikz/3d/circle on sphere/fill` (initially `empty`)
Style of the disk of the circle, which is painted first, e.g. `fill/.style={fill=red,fill opacity=0.2}`.
- `/tikz/3d/circle on sphere/start angle= $\langle angle \rangle$` (no default, initially 0)
Start of the arc. Angles are measured in the plane of the circle from the projection of `u`, positively around `n`. The arc gets split where it passes from the front of the sphere to the back.
- `/tikz/3d/circle on sphere/end angle= $\langle angle \rangle$` (no default, initially 360)
End of the arc.
- `/tikz/3d/circle on sphere/u` (initially (1,0,0))
Direction of the angle 0 (it gets projected into the plane of the circle).
- `/tikz/3d/circle on sphere/start color= $\langle color \rangle$` (no default, initially `empty`)
If set, the arc gets a color gradient from $\langle color \rangle$ at its start to `end color` at its end. It is drawn as sub-arcs of at most `segment angle` degrees, each with its own color and, in a scene, its own depth. Only the last sub-arc keeps the arrow tips, so put an end tip (e.g. `->`) in the styles of the parts.
- `/tikz/3d/circle on sphere/end color= $\langle color \rangle$` (no default, initially `empty`)
Color at the end of the arc (initially the start color).
- `/tikz/3d/circle on sphere/segment angle= $\langle angle \rangle$` (no default, initially 360)
In a scene, the arc is painted as cells of at most this angle (on a grid that starts at the point closest to the viewer), each with the depth of its centroid. Sub-arcs of 1pt or more get round caps, which close their joints.

/tikz/3d/circle on sphere/gradient angle= $\langle angle \rangle$ (no default, initially 2)

Largest angle of a sub-arc of a color gradient.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,theta=70},
declare function={R=2;}]
\shade[ball color=white,3d/screen coords] circle[radius=R];
\path pic{3d/circle on sphere={R=R,P={0.5,-1.2,1}}};
\end{tikzpicture}
```



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,theta=70},
declare function={R=2;}]
\shade[ball color=white,3d/screen coords,opacity=0.5]
circle[radius=R];
\path pic{3d/circle on sphere={R=R,latitude=30,
fill/.style={fill=red,fill opacity=0.2}}}
pic{3d/circle on sphere={R=R,longitude=40}}
pic{3d/circle on sphere={R=R,n={(1,1,0)},end angle=270,
start color=blue,end color=red,
/tikz/3d/visible/.style={thick,->},
/tikz/3d/hidden/.style={thick,->,opacity=0.4}}};
\end{tikzpicture}
```

/tikz/pics/3d/sphere= $\langle options \rangle$ (no default, initially empty)

A sphere of radius R around the pic position, painted with the style **shell** and outlined with **outline**. In a scene the shell is painted twice, behind and in front of everything inside the sphere, and the visible parts of **3d/circle on sphere** go on top of it; for a translucent sphere, the opacity of **shell** applies to each layer.

/tikz/3d/sphere/R (initially 1)

Radius.

/tikz/3d/sphere/shell (initially ball color=white)

Style of the shell.

/tikz/3d/sphere/outline (initially draw)

Style of the outline.

/tikz/pics/3d/point on sphere= $\langle options \rangle$ (no default, initially empty)

Puts a node at the point P on the sphere around C , with the style **front** and the text **front text** if the point faces the viewer, and with **back** and **back text** otherwise.

/tikz/3d/point on sphere/C (initially (0,0,0))

Center of the sphere.

/tikz/3d/point on sphere/P (initially (0,0,1))

The point.

/tikz/3d/point on sphere/front text (initially \textbullet)

Text of the node on the front.

/tikz/3d/point on sphere/back text (initially \(\circ\))

Text of the node on the back.

`/tikz/pics/3d/circle=<options>` (no default, initially empty)

A circle (or an arc) around the position of the pic, split into the half that is closer to the viewer, drawn with `front`, and the other half, drawn with `back`. In a scene (section 1.5) the halves are separate pieces, so the circle can wrap around other objects. The keys `n`, `u`, `start angle`, `end angle`, `fill`, `start color`, `end color`, `segment angle` and `gradient angle` work as for `3d/circle` on sphere.

`/tikz/3d/circle/r` (initially 1)

Radius.

`/tikz/3d/circle/front` (initially empty)

Style of the half closer to the viewer.

`/tikz/3d/circle/back` (initially empty)

Style of the other half.

`/tikz/pics/3d incircle=<options>` (no default, initially empty)

Inscribes a circle in a triangle in 3 dimensions.

`/tikz/3d incircle/A` (initially (1,0,0))

First coordinate. Can be either symbolic or explicit.

`/tikz/3d incircle/B` (initially (0,1,0))

Second coordinate, like above.

`/tikz/3d incircle/C` (initially (0,0,1))

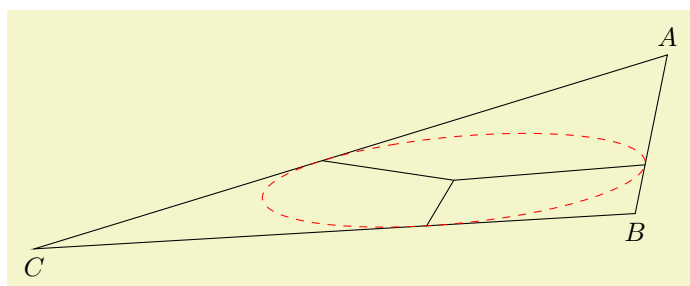
Third coordinate, like above.

`/tikz/3d incircle/center name` (initially I)

Name of the center coordinate that will be derived.

`/tikz/3d incircle/projection prefix` (initially `tmpp`)

In TikZ the coordinates are global. The code for the circle is more comprehensible if named coordinates are introduced. Their names will begin with this prefix. Changing the prefix allows users to avoid overwriting existing coordinates.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=110,psi=0,theta=70}]
\draw
  (8,5,5) coordinate[label=above:{$A$}] (A) --
  (1,2,0) coordinate[label=below:{$B$}] (B) --
  (5,-5,0) coordinate[label=below:{$C$}] (C) -- cycle;
\path pic[red,dashed]{3d incircle={%
  A={A},B={B},C={C},center name=I}};
\draw (I) -- (tmppa) (I) -- (tmppb) (I) -- (tmppc);
\end{tikzpicture}
```

`/tikz/3d/polyhedron/draw face with corners=<list of corners>` (no default, initially empty)

Draws a face of a polyhedron through the specified corners. It distinguishes between “visible” and “hidden” faces. The user can specify a point inside the corner. Then the outwards pointing normal of the face can either have a negative projection on the screen, in which case the face is hidden, or else it is visible.

`/tikz/3d/polyhedron/draw open face with corners=<list of corners>` (no default, initially `empty`)
 Same as the previous key except it does not close the surface boundary path.

`/tikz/3d/polyhedron/O=<coordinate>` (no default, initially `(0,0,0)`)
 Coordinate inside the polyhedron.

`/tikz/3d/polyhedron/L=<vector>` (no default, initially the value of `3d/light`)
 Direction towards the light, used for the faces of polyhedra only. By default it follows `/tikz/3d/light`.

`/tikz/3d/polyhedron/shading function=<name>` (no default, initially `tikztdpolyhedronshade`)
 Function of the cosine between the outward normal of the face and the light direction. It returns the percentage of the face color that is mixed with black. The default, `tikztdpolyhedronshade(\x)=100*tikztdlight(\x)` uses the lighting model below. The look of earlier versions is obtained with `declare function={tikztdpolyhedronsh`

`/tikz/3d/light=<vector>` (no default, initially `(1,1,1)`)
 Direction pointing towards a single directional light, in the coordinates in which the drawing is specified (the frame of `3d/install view`). The light is thus fixed in the scene: changing the view changes what is lit as seen from the camera. It is used by polyhedra, `cylinder`, `ycylinder` and `3d/shaded cone`. A surface element with unit outward normal n and the unit light direction L gets the intensity $I = \min(1, a + d \max(0, n \cdot L))$ (Lambert), and the color `<color>!100I!black`. The pgf function `tikztdlight` computes I from $n \cdot L$.

`/tikz/3d/light/ambient=<number>` (no default, initially `0.5`)
 The ambient part a .

`/tikz/3d/light/diffuse=<number>` (no default, initially `0.5`)
 The diffuse part d .

`/tikz/3d/polyhedron/fore layer=<layer>` (no default, initially `main`)
 Layer of the visible faces.

`/tikz/3d/polyhedron/back layer=<layer>` (no default, initially `main`)
 Layer of the hidden faces.

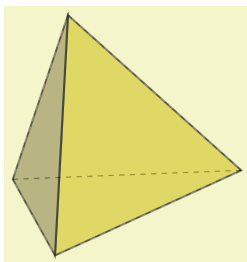
`/tikz/3d/polyhedron/color=<color>` (no default, initially `yellow`)
 Color of the polyhedron.

`/tikz/3d/polyhedron/fore=<style>` (no default, initially `draw,solid`)
 Style for visible faces. One can turn off the fill by adding `fill=none`.

`/tikz/3d/polyhedron/back=<style>` (no default, initially `draw,dashed,fill opacity=0`)
 Style for visible faces.

`/tikz/3d/polyhedron/complete dashes=<style>` (no default, initially `on 2pt off 2pt`)
 Switches on dashes for the edges which start and end with an on phase.

`/tikz/cheating dash=<style>` (no default, initially `on 2pt off 2pt`)
 Switches on dashes with an on phase. Taken from [Mark Wibrow's answer](#).



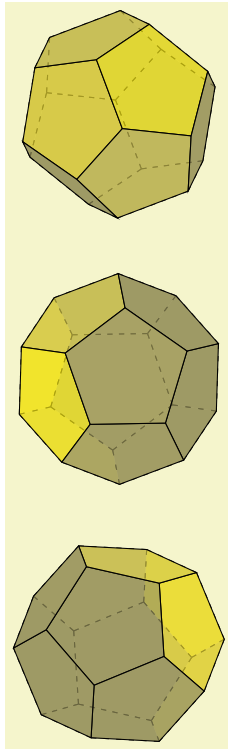
```
\usetikzlibrary {3dtools}
\pgfdeclarelayer{background}
\pgfdeclarelayer{foreground}
\pgfsetlayers{background,main,foreground}
\begin{tikzpicture}
\begin{scope}[3d/install view={phi=110,psi=10,%
theta=70},scale=2]
\path (0,0,1) coordinate (v1)
({sqrt(8/9)},0,-1/3) coordinate (v2)
({-sqrt(2/9)},{sqrt(2/3)},-1/3) coordinate (v3)
({-sqrt(2/9)},{-sqrt(2/3)},-1/3) coordinate (v4);
\tikzset{3d/polyhedron/.cd,
fore layer=foreground,back layer=background,
fore/.append style={opacity=0.4,thick},
back/.append style={3d/polyhedron/complete dashes,
opacity=0.4},
draw face with corners={{(v1)},{(v2)},{(v3)}}},
draw face with corners={{(v1)},{(v2)},{(v4)}}},
draw face with corners={{(v1)},{(v3)},{(v4)}}},
draw face with corners={{(v2)},{(v3)},{(v4)}}}}
\end{scope}
\end{tikzpicture}
```

`/tikz/3d/define vertices=<list>` (no default, initially empty)

Allows the user to define a list of vertices from a list. These vertices have names that consist of their indices in the list. One can work with the `name prefix` key to give them more meaningful names. Such lists can be generated with *Mathematica*, e.g. with `N[PolyhedronData["Dodecahedron", "VertexCoordinates"]]`.

`/tikz/3d/polyhedron/create faces from vertex list=<list>` (no default, initially empty)

Draws a face from a list of vertices. Such lists can be generated with *Mathematica*, e.g. with `PolyhedronData["Dodecahedron", "FaceIndices"]`. This key is very similar to `draw face with corners` except that it also adds the `name prefix`, and that it loops over the list. One should avoid spaces in the list.



```
\usetikzlibrary {3dtools}
\pgfdeclarelayer{background}\pgfdeclarelayer{foreground}
\pgfsetlayers{background,main,foreground}
\begin{tikzpicture}
\edef\lstV{{-1.376,0.,0.2628},{1.376,0.,-0.2628},{-0.4253,-1.309,0.2628},
{-0.4253,1.309,0.2628},{1.113,-0.8090,0.2628},{1.113,0.8090,0.2628},
{-0.2628,-0.8090,1.113},{-0.2628,0.8090,1.113},{-0.6881,-0.5,-1.113},
{-0.6881,0.5,-1.113},{0.6881,-0.5,1.113},{0.6881,0.5,1.113},
{0.8506,0.,-1.113},{-1.113,-0.8090,-0.2628},{-1.113,0.8090,-0.2628},
{-0.8506,0.,1.113},{0.2628,-0.8090,-1.113},{0.2628,0.8090,-1.113},
{0.4253,-1.309,-0.2628},{0.4253,1.309,-0.2628}}
\edef\lstFaces{{15,10,9,14,1},{2,6,12,11,5},{5,11,7,3,19},{11,12,8,16,7},
{12,6,20,4,8},{6,2,13,18,20},{2,5,19,17,13},{4,20,18,10,15},{18,13,17,9,10},
{17,19,3,14,9},{3,7,16,1,14},{16,8,4,15,1}}
\tikzset{3d/polyhedron/.cd,fore layer=foreground,back layer=background,
fore/.append style={fill opacity=0.7},
back/.append style={3d/polyhedron/complete dashes,fill opacity=0.7}}
\begin{scope}[3d/install view={phi=120,psi=20,theta=70}]
\tikzset{name prefix=Va,%<- used for all vertices
3d/define vertices/.expanded={\lstV},
3d/polyhedron/create faces from vertex list/.expanded={\lstFaces}}
\end{scope}
\begin{scope}[yshift=-3.5cm,
3d/install view={phi=210,psi=30,theta=60}]
\tikzset{name prefix=Vb,%<- used for all vertices
3d/define vertices/.expanded={\lstV},
3d/polyhedron/create faces from vertex list/.expanded={\lstFaces}}
\end{scope}
\begin{scope}[yshift=-7cm,
3d/install view={phi=30,psi=-10,theta=75}]
\tikzset{name prefix=Vc,%<- used for all vertices
3d/define vertices/.expanded={\lstV},
3d/polyhedron/create faces from vertex list/.expanded={\lstFaces}}
\end{scope}
\end{tikzpicture}
```

The library has also some experimental pics for cones, truncated cones and cylinders. They share a couple of common keys.

`/tikz/3d/r` (initially 1)

Key for radii, e.g. of cylinders or cones.

`/tikz/3d/h` (initially 1)

Key for heights, e.g. of cylinders or cones.

`/tikz/3d/R` (initially 2)

Key for larger radii, e.g. of a frustum.

`/tikz/pics/3d/visible` (initially draw,solid)

Style for visible lines.

`/tikz/pics/3d/hidden` (initially draw,very thin,cheating dash)

Style for hidden lines.

`/tikz/pics/3d/cone` (initially empty)

Draws a cone whose base, of radius `r`, is centered at the pic position and whose apex sits at distance `h` along `3d/cone/axis`. The pic defines the coordinates `-base`, `-apex` and, for compatibility, `-tip` (same as `-apex`). Hidden parts of the base rim are drawn with `3d/hidden`.

`/tikz/3d/cone/axis` (initially (0,0,1))

Direction of the cone axis (it gets normalized). Also used by `3d/frustum`.

`/tikz/3d/cone/apex=(3d vector)` (no default)

Apex relative to the base center. Sets `3d/cone/axis` and `3d/h`.

/tikz/3d/cone/outer (initially empty)

Style of the visible part of the mantle, e.g. `fill=red!30`.

/tikz/3d/cone/inner (initially empty)

Style of the base region when the base side faces the viewer (the base disk, or the interior of an open cone).

/tikz/3d/cone/outer shade (initially `3d/cone/lit side=outer`) (no default)

Shading of the mantle used by `3d/shaded cone`. It is centered at the apex. By default the mantle is lit by `3d/light`.

/tikz/3d/cone/inner shade (initially `3d/cone/lit side=inner`) (no default)

Shading of the base region (the interior of the open cone) used by `3d/shaded cone`, lit by `3d/light`.

/tikz/3d/cone/color=*<color>* (no default, initially white)

Color of the lit cone.

/tikz/3d/cone/lit side=*<outer or inner>* (no default)

Sets `3d/cone/lit/max` and `3d/cone/lit/min` to the intensities of the brightest and darkest visible generator of that side and calls `3d/cone/light shading` with the screen angle, seen from the apex, of the brightest generator.

/tikz/3d/cone/light shading=*<angle>* (no default)

The shading hook. The default declares a functional shading in the angle around the apex that reproduces the exact Lambert intensity of every visible generator (brightness is constant along a generator). Replace it to use a different shading, e.g. one built from *<angle>* and the two keys above.

/tikz/3d/cone/screen angle (no value)

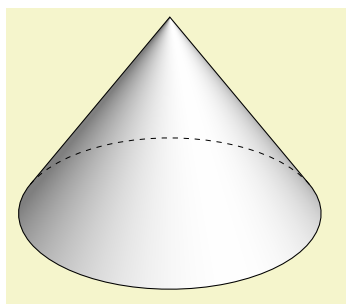
Set by the cone pics before the shading styles are used: the screen angle of the direction from the base center to the apex. For instance, `outer shade/.style={3d/cone shading={alpha/.evaluated={\pgfkeysvalueof{tikz/3d/cone/screen angle}}}}` rotates the shading with the cone.

/tikz/pics/3d/shaded cone (initially empty)

Draws a shaded cone with the basis in the current xy plane.

/tikz/3d/cone shading=*<options>* (no default)

Defines and uses a functional shading that can be used for cones. The parameters are the coordinates of the tip *x* and *y*, which have to satisfy $-25 < x, y < 25$, and *alpha*, which indicates the rotation of the shading.



```
\usetikzlibrary {calc,3dtools}
\begin{tikzpicture}[3d/install view=%,
{phi=110,psi=0,theta=60}]
\pic{3d/shaded cone={r=2,h=3}};
\end{tikzpicture}
```

/tikz/pics/3d/frustum (initially empty)

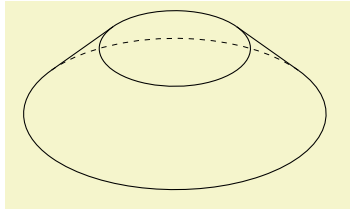
Draws a frustum with the basis in the current xy plane, open at both ends. The generatrices of the silhouette divide the mantle in two patches: one shows its outer side to the viewer, painted with `3d/frustum/outer`, the other its inner side, painted with `3d/frustum/inner`. The rims and the generatrices are drawn with `3d/visible` and `3d/hidden`. Notice that if both radii are equal this is just a cylinder.

`/tikz/3d/frustum/outer` (initially empty)

Style of the patch of the mantle that shows its outer side.

`/tikz/3d/frustum/inner` (initially empty)

Style of the patch of the mantle that shows its inner side, e.g. the coating of a conical mirror.



```
\usetikzlibrary {calc,3dtools}
\begin{tikzpicture}[3d/install view=
{phi=110,psi=0,theta=60}]
\pic{3d/frustum};
\end{tikzpicture}
```

`/tikz/pics/ycylinder` (initially empty)

A cylinder in the y -direction. This pic requires the `calc` library. As of now it does only work for $\psi=0$. This key is obsolete because of the `cylinder` pic which allows the user to dial an arbitrary axis.

`/tikz/pics/cylinder` (initially empty)

A cylinder with an arbitrary axis. This pic requires the `calc` library. Seen along its axis it reduces to the cap facing the viewer.

`/tikz/pics/3d/cylinder/axis` (initially $(0,0,1)$)

Axis of cylinder.

`/tikz/pics/3d/cylinder/mantle` (initially draw)

Style for cylinder mantle. If it contains no fill or shading, the mantle shows the lighting of `3d/cylinder/lighting`. A user shading (e.g. `left color`, `right color`, `middle color`) is oriented across the axis.

`/tikz/3d/cylinder/color= $\langle color \rangle$` (no default, initially white)

Color of the lit mantle and caps.

`/tikz/3d/cylinder/lighting` (initially a path picture with the lit shading)

Painted below the mantle and the cap styles. In parallel projection the Lambert intensity of the mantle only depends on the position across the silhouette, so the default axial shading is exact. A cap is flat: the visible one, with outward normal $n = \pm \text{axis}$, gets the constant intensity $\min(1, a + d \max(0, n \cdot L))$. It is not painted below a mantle or cap whose style fills or shades. Set it to `{}` to switch the lighting of mantle and caps off.

`/tikz/3d/cylinder/hole= $\langle radius \rangle$` (no default, initially 0)

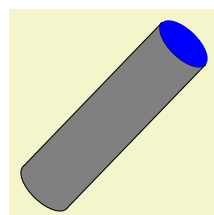
Radius of a hole in the caps.

`/tikz/3d/cylinder/open= $\langle boolean \rangle$` (no default, initially false)

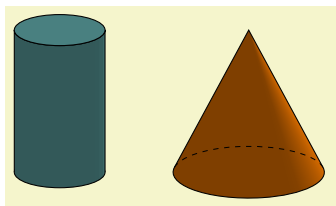
A tube without caps: only the rim of the cap facing the viewer is drawn.

`/tikz/pics/3d/cylinder/top` (initially draw)

Style for the visible cap (top or bottom, also used by `ycylinder`). If it contains no fill, the cap shows the lighting of `3d/cylinder/lighting`.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}[3d/install view={phi=30,psi=0,theta=80}]
\pic{cylinder={r=0.4,h=3,
cylinder/axis={(1,0,1)},
cylinder/top/.style={fill=blue}}};
\end{tikzpicture}
```

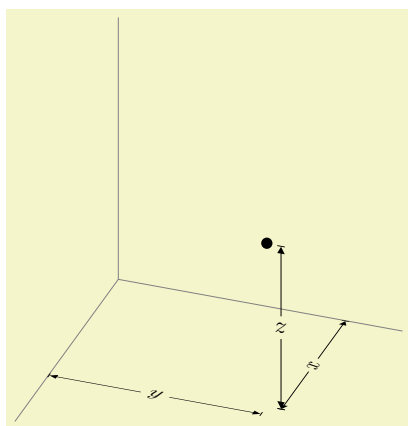


```
\usetikzlibrary {calc,3dtools}
\begin{tikzpicture}[3d/install view={phi=110,psi=0,theta=70},
3d/light={(-1,0.3,0.5)},3d/cylinder/color=teal!60,
3d/cone/color=orange]
\pic{cylinder={r=0.6,h=2}};
\begin{scope}[xshift=2.5cm]
\pic{3d/shaded cone={r=1,h=2}};
\end{scope}
\end{tikzpicture}
```

/tikz/pics/3d/dim line

(initially empty)

Draws a line that allows users to annotate axes and/or indicate dimensions. It expects the user to specify the start and end points, A and B, as well as the normal $\mathbf{n}$ of the plane the line is to be drawn. In addition, the $\mathbf{d}$ key, which can take negative values, determines the distance of the line from A--B. The label is given by the $\mathbf{l}$ key, and the `dim label` style controls its appearance. This pic requires the `calc` library.



```
\usetikzlibrary {calc,3dtools}
\begin{tikzpicture}[3d/install view={phi=110,theta=60},
declare function={a=4;rx=2.5;ry=3;rz=2.5;}]
\draw[gray,help lines] (0,0,0) coordinate (0)
-- (a,0,0)
(0) -- (0,a,0) (0) -- (0,0,a);
\path (rx,ry,rz) coordinate[coordinate[circle,inner sep=1.5pt,
fill] (r);
\path pic{3d/dim line={A={(0,ry,0)},B={(rx,ry,0)},l={\$x\$}}}}
pic{3d/dim line={A={(rx,0,0)},B={(rx,ry,0)},l={\$y\$},d=-2mm}}
pic{3d/dim line={A={(rx,ry,0)},B={(r)},n={(1,0,0)},l={\$z\$},
d=-2mm,dim label/.append style={rotate=90}}}};
\end{tikzpicture}
```

To do:

- transform to plane given by three non-degenerate coordinates
- transform to plane given by normal and one point
- maybe layering/visibility

1.8 Path management

This is poor marmots version of path management. Almost certainly `spath3` is way superior. The idea is that one can store paths, and redraw them, possibly in reversed order. This recording tool uses the `show path construction` decoration from the `decorations.pathreplacing` library. In order to make use of these tools, you need to load the library.

/tikz/store path= $\langle name \rangle$

(no default, initially empty)

Records a path under the name $\langle name \rangle$. If you want to draw and record a path at the same time, use `postaction={store path= $\langle name \rangle$ }`.

/tikz/stored path/coordinate prefix= $\langle prefix \rangle$

(no default, initially stored-)

Prefix for the stored coordinates. If you store several paths and want to possibly combine them, you may want to give each path a unique prefix.

/tikz/stored path/reset coordinate index

(no value)

Resets the counter for the stored coordinates. Mainly for internal use.

/tikz/stored path/restore path= $\langle name \rangle$

(no default, initially empty)

Restores a saved path of name $\langle name \rangle$ using the `insert path` key.

/tikz/stored path/restore reversed path=*<name>* (no default, initially empty)
Restores a saved path of name *<name>* using the `insert path` key in *reversed* order.

/tikz/stored path/append path=*<name>* (no default, initially empty)
Restores a saved path of name *<name>* using the `insert path` key with a `--` at the beginning. This may be necessary to patch paths together.

/tikz/stored path/append reversed path=*<name>* (no default, initially empty)
Restores a saved path of name *<name>* using the `insert path` key in *reversed* order with a `--` at the beginning. This may be necessary to patch paths together.

/tikz/stored path/first coordinate of=*<name>* (no default, initially empty)
Inserts the first coordinate of the path named *<name>* using the `insert path` key.

/tikz/stored path/last coordinate of=*<name>* (no default, initially empty)
Inserts the last coordinate of the path named *<name>* using the `insert path` key.

/tikz/stored path/coordinate with index=*n* of *<name>* (no default, initially empty)
Inserts the n^{th} coordinate of the path named *<name>* using the `insert path` key. Notice that at present there is no sanity check. If *n* is larger than the number of stored coordinates, there will be an error.

tikztdindexoflastcoordinate

Yields index of the last coordinate of a stored path. This can be used to test if a path is empty.

1.9 List management

3d ordering has a lot to do with sorting lists. Here are some poor marmot's tools to deal with such lists and some primitive versions of associative arrays. First of all, there are a couple of *key handlers* that allow one to add and remove items. Notice that, in order to apply key handlers to a list, the list has to be stored in a *<key>*.

Key handler *<key>*/.add item

Key handler that adds an item to a list *<key>*.

Key handler *<key>*/.remove item

Key handler that removes an item from a list *<key>*.

Key handler *<key>*/.count=*<macro>*

Key handler that stores the number of elements of a given list *<key>* in a macro, which is the mandatory argument.

Key handler *<key>*/.print list

Key handler that prints a list. It threads `/list management/print item` code over the list, and separates the items by `/list management/list separators`.

/list management/print item (no value)
Code used by the `.print list` key handler.

/list management/list separator (initially ,)
Code used by the `.print list` key handler to separate items.

The first 9 TikZlings are {anteater, bear, bee, cat, coati, hippo, koala, marmot, mouse}.

After the wolpertinger joined, they were 1, and the list became {anteater, bear, bee, cat, coati, hippo, koala, marmot, mouse, wolpertinger}.

Sadly, the mouse could not stand the wolpertinger, so the list TikZlings got reduced to 9, consisting of {anteater, bear, bee, cat, coati, hippo, koala, marmot, wolpertinger}.

```
\usetikzlibrary {3dtools}
\begin{minipage}{6.4cm}
\pgfkeys{/my lists/.cd,
tikzlings/.initial={anteater,%
bear,bee,cat,coati,%
hippo,koala,marmot,mouse}}
The first\
\pgfkeys{/my lists/tikzlings/.count=\myn}\myn\
Ti\emph{k}Zlings are\
\{\pgfkeys{/my lists/tikzlings/.print list}\}.
\par\medskip
\pgfkeys{/my lists/tikzlings/.add item=wolpertinger}
After the wolpertinger joined, they were \pgfkeys{/my
lists/tikzlings/.count=\myn}{\myn$}, and the list became\
\{\pgfkeys{/my lists/tikzlings/.print list}\}.
\par\medskip
\pgfkeys{/my lists/tikzlings/.remove item=mouse}
Sadly, the mouse could not stand the wolpertinger,\
so the list Ti\emph{k}Zlings got reduced to\
\pgfkeys{/my lists/tikzlings/.count=\myn}{\myn$},\
consisting of\
\{\pgfkeys{/my lists/tikzlings/.print list}\}.
\end{minipage}
```

Key handler $\langle key \rangle/.is array=\langle list \rangle$

Key handler makes the current key $\langle key \rangle$ an array consisting of $\langle list \rangle$. You need to use a new key for that, i.e. you cannot overwrite existing keys. The benefit is that you can access the items in the list simply via $\langle key \rangle/n$, where n is an integer-valued index, and that $\langle key \rangle/\dim$ contains its dimension.

Key handler $\langle key \rangle/.sort numeric list=\langle macro \rangle\langle macro \rangle$

Key handler that sorts a list $\langle key \rangle$. The resulting sorted list is stored in the first argument, which has to be a macro, and the permutation is stored in the second argument, which needs to be a macro as well. This allows one to know the position of a given entry in a sorted list.

Often one is given a list of entries where one wants to sort on the basis of some function that can be fed with an entry. One may also want to have an index mapping which indicates at which position the i^{th} entry of the initial list ended up getting. In this example, we start out with

$$\{\lambda_i\}_{i=1}^n = \{12, 19, 1, 3, 17, 4, 23, 5, 9, 7\}.$$

The fact that we used the `/is array` key handler means that we know its dimension, $n = 10$. After sorting we get

$$\{\lambda'_i\}_{i=1}^n = \{1, 3, 4, 5, 7, 9, 12, 17, 19, 23\}$$

and an index or reshuffling or permutation list

$$P = (3, 4, 6, 8, 10, 9, 1, 5, 2, 7).$$

For instance, the 5th entry of the permutation list being $P_5 = 10$ tells us that the 10th entry of our original list, $\lambda_{10} = 7$, ended up at position 5 in the sorted list, i.e. $\lambda'_5 = \lambda_{10}$. Similarly, $P_3 = 6$ means that for the 6th entry of our original list, $\lambda_6 = 4 = \lambda'_3$.

```
\usetikzlibrary {3dtools}
\begin{minipage}{6.4cm}
\pgfkeys{/my lists/.cd,
my initial array/.is array={%
12,19,1,3,17,4,23,5,9,7},
my values/.initial=\pgfkeysvalueof{%
/my lists/my initial array/content},%
my values/.sort numeric list=%
{\temp}{\templ},%
my sorted array/.is array/.expanded={\temp},
my index machinery/.is array/.expanded={\templ}}%
Often one is given a list of entries where\
one wants to sort on the basis of some\
function that can be fed with an entry.\
One may also want to have an index mapping\
which indicates at which position the\
 $\mathrm{th}$  entry of the initial list\
ended up getting. In this example, we\
start out with
\[\{\lambda_i\}_{i=1}^n=
\{\pgfkeys{/my lists/my initial array/%
content/.print list}\}
\;.]
The fact that we used the \texttt{/is array}\
key handler means that we know its dimension,\
 $n=\pgfkeysvalueof{/my lists/my initial array/dim}$ .\
After sorting we get
\[\{\lambda_i'\}_{i=1}^n=
\{\pgfkeys{/my lists/my sorted array/%
content/.print list}\}
\]
and an index or reshuffling or permutation list
\[P=(\pgfkeys{/my lists/%
my index machinery/content/.print list})\;.]
For instance, the  $\mathrm{th}$  entry of the\
permutation list being\
 $P_5=\pgfkeysvalueof{/my lists/my index machinery/5}$ \
tells us that the\
 $\pgfkeysvalueof{/my lists/my index machinery/5}$ \
 $\mathrm{th}$  entry of our original list,\
 $\lambda_{\pgfkeysvalueof{/my lists/%
my index machinery/5}}$ \
 $=\pgfkeysvalueof{/my lists/my initial array/%
\pgfkeysvalueof{/my lists/my index machinery/5}}$ ,\
ended up at position  $5$  in the sorted list,\
i.e.  $\lambda_5'=\lambda_{\pgfkeysvalueof{/my lists/%
my index machinery/5}}$ . Similarly,\
 $P_3=\pgfkeysvalueof{/my lists/my index machinery/3}$ \
means that for the\
 $\pgfkeysvalueof{/my lists/my index machinery/3}$ \
 $\mathrm{th}$  entry of our original list,\
 $\lambda_{\pgfkeysvalueof{/my lists/%
my index machinery/3}}$ \
 $=\pgfkeysvalueof{/my lists/my initial array/%
\pgfkeysvalueof{/my lists/my index machinery/3}}$ \
 $=\lambda_3'$ .
\end{minipage}
```

1.10 Miscellaneous goodies

`/tikz/on layer= $\langle layer \rangle$` (no default)

Allows one to set a layer in a path. Taken from <https://tex.stackexchange.com/a/20426>.

`/tikz/node on layer= $\langle layer \rangle$` (no default)

Allows one to set a layer of a node in a path. Taken from <https://tex.stackexchange.com/a/20426>.

`/tikz/even odd clip` (no value)

Enables the ‘even odd rule’ for clips.

`/tikz/save named path` (no value)

Saves a path very much like the `save path` key but uses strings, i.e. you can say `save named path=A`, and prefixes the global macro by `tikz@td@named@path@` to minimize the chance that this macro interferes with other macros.

`/tikz/use named path` (no value)

Allows one to use a path saved with `save named path`, e.g. `use named path=A`.

`/tikz/same bounding box=id` (default A)

Allows one to synchronize the bounding box of various pictures. Each picture with the same *id* will have the same bounding box after the second compilation.

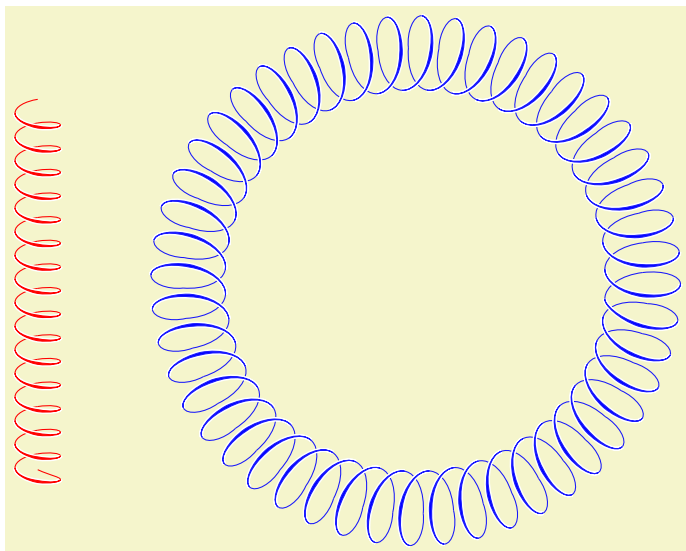
1.11 3D-like decorations

`/tikz/decorations/3d complete coil` (no value)

3d-like coil where the front is thicker than the back.

`/tikz/decorations/3d coil closed` (no value)

Indicates that the coil is closed.



```
\usetikzlibrary {3dtools}
\begin{tikzpicture}
\draw[decoration={3d coil color=red,aspect=0.35, segment length=3.1mm,
amplitude=3mm,3d complete coil},
decorate] (0,1) -- (0,6);
\draw[decoration={3d coil color=blue,3d coil opacity=0.9,aspect=0.5,
segment length={2*pi*3cm/50}, amplitude=5mm,3d complete coil,
3d coil closed},
decorate] (5,3.5) circle[radius=3cm];
\end{tikzpicture}
```

1.12 Circles of a sphere (compatibility library)

`\usetikzlibrary{3dtools.circleofsphere}` provides the commands of the `tikz-3dplot-circleofsphere` package by Matthias Wolff, drawn with `3d/circle on sphere` and `3d/point on sphere`, so they work with any view and inside scenes. Load it instead of the package. The styles `tdplotCsFront`, `tdplotCsBack`, `tdplotCsFill`, `tdplotPtFront` and `tdplotPtBack` keep their meaning; `tdplotCsDrawAux` is not supported. `\tdplotCsComputeTransformRotScreen` needs `tikz-3dplot`.

`\tdplotCsDrawCircle[<style>]{<radius>}{<alpha>}{<beta>}{<elevation>}`

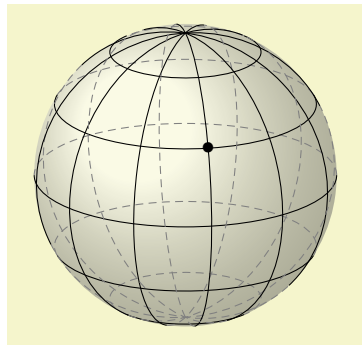
The circle whose plane has the normal with azimuth *<alpha>* and polar angle *<beta>*, at the elevation angle *<elevation>* above the center. The circle is filled only if *<style>* mentions `tdplotCsFill`.

`\tdplotCsDrawGreatCircle[ $\langle style \rangle$ ]{ $\langle radius \rangle$ }{ $\langle alpha \rangle$ }{ $\langle beta \rangle$ }`

`\tdplotCsDrawLatCircle[ $\langle style \rangle$ ]{ $\langle radius \rangle$ }{ $\langle latitude \rangle$ }`

`\tdplotCsDrawLonCircle[ $\langle style \rangle$ ]{ $\langle radius \rangle$ }{ $\langle alpha \rangle$ }`

`\tdplotCsDrawPoint[ $\langle style \rangle$ ]{ $\langle radius \rangle$ }{ $\langle alpha \rangle$ }{ $\langle beta \rangle$ }`



```
\usetikzlibrary {3dtools.circleofsphere}
\begin{tikzpicture}[3d/install view={phi=110,theta=70},
  tdplotCsBack/.style={densely dashed,gray}]
  \shade[ball color=white,3d/screen coords,opacity=0.5]
    circle[radius=2];
  \foreach \a in {-60,-30,...,60} {\tdplotCsDrawLatCircle{2}{\a}}
  \foreach \a in {0,30,...,150} {\tdplotCsDrawLonCircle{2}{\a}}
  \tdplotCsDrawPoint{2}{30}{60}
\end{tikzpicture}
```

1.13 Remarks on the implementation

When the library gets loaded, the version of TikZ gets checked. If the version is earlier than 3.1.1, a warning is issued. In this case, one cannot use the library unless one updates the pgf installation. Notice that many features of this library depend on TikZ's internal macro `\tikz@dc1@coord@`. If this macro gets removed or changed, many features of the library will no longer work. The tracking of depths (section 1.1) wraps `\pgfpointxyz`, `\pgfpointxy`, `\pgfpointpolarxy`, `\pgfsetxvec` and its relatives, the `shift` key and several internal macros of TikZ and of its `calc` library (the current point, named coordinates, node placement, and the `($...$)` parser). It costs about 1–2% of the run time of pictures that consist of many plain 2d paths and nodes. So this whole setup is fragile. Of course, this complication is not unheard of, but it might be important to know about this before deciding to use this library.

Acknowledgments

I'd like to thank [minhthien2016](#) for continuous support and encouragement. I thank the creators, maintainers, moderators and users of [topanswers.xyz](#) for their invaluable support. These acknowledgements extend to all users on this site, including those with whom I have passionate disagreements. Disagreements are good, they indicate critical thinking, or in the words of Nobel Laureate Steven Weinberg, physics thrives on contradictions.